

Timothy Song

Professor Shim

DATA 260

5 October 2026

Homework 5 Report

GitHub Repository: <https://github.com/TimothyYSong/data260-9981>

Section 0 - Personal Configuration and Domain

Hardware: MacBook Pro M4 Pro, 14 cores (10 performance and 4 efficiency), 24 GB memory

Model: qwen3:8b

Tagged commit hash: 276872ed8c5af1dea89b483f2ad1f5ba1a861c5e

SID4: 9981

PORT_BASE: 8081

PREFIX: s9981

SEED: 9981

VERIFY_SEED: 269981

DOMAIN_ID: 5 (Local restaurant inspections)

Part 1

I. Database

- I created a new Restaurant table as the related entity for my Local Restaurant Inspections domain. A restaurant can have multiple inspection records. The table includes an auto-incrementing primary key, restaurant name, address, a unique permit code, and creation/update timestamps.

```
models.py M X
code > models.py > InspectionNote
9   class Restaurant(Base):
10       __tablename__ = "restaurants"
11
12       id = Column(Integer, primary_key=True, index=True, autoincrement=True)
13       name = Column(String(255), nullable=False)
14       address = Column(String(255), nullable=False)
15       permit_code = Column(String(100), unique=True, nullable=False, index=True)
16       created_at = Column(DateTime, nullable=False, default=datetime.utcnow)
17       updated_at = Column(
18           DateTime,
19           nullable=False,
20           default=datetime.utcnow,
21           onupdate=datetime.utcnow,
22       )
```

- I extended the existing RestaurantInspection entity. Each inspection now has a unique inspection_code, a numeric violation_count with a default value of 0, a foreign key restaurant_id that links the inspection to the related Restaurant entity, and created_at / updated_at timestamps. The relationship is one-to-many: one restaurant can have multiple inspection records.

```
models.py M X
code > models.py > User
30 class RestaurantInspection(Base):
31     __tablename__ = "restaurant_inspections"
32
33     id = Column(Integer, primary_key=True, index=True, autoincrement=True)
34     restaurant_name = Column(String(255), nullable=False)
35     restaurant_address = Column(String(255), nullable=False)
36
37     inspection_code = Column(
38         String(100),
39         unique=True,
40         nullable=False,
41         index=True,
42     )
43
44     violation_count = Column(
45         Integer,
46         nullable=False,
47         default=0,
48     )
49
50     restaurant_id = Column(
51         Integer,
52         ForeignKey("restaurants.id"),
53         nullable=False,
54     )
55
56     created_at = Column(
57         DateTime,
58         nullable=False,
59         default=datetime.utcnow,
60     )
61
62     updated_at = Column(
63         DateTime,
64         nullable=False,
65         default=datetime.utcnow,
66         onupdate=datetime.utcnow,
67     )
68
69     restaurant = relationship(
70         "Restaurant",
71         back_populates="inspections",
72     )
73
74     notes = relationship(
75         "InspectionNote",
76         back_populates="restaurant_inspection",
77     )
```

- The User model stores a password_hash rather than a plain-text password, and authentication verifies submitted passwords against the stored bcrypt hash. For the related Restaurant entity, the restaurant_id foreign key uses ondelete="RESTRICT" so a restaurant cannot be deleted while associated RestaurantInspection records still exist.

```
models.py M X
code > models.py > InspectionNote
80 class User(Base):
81     __tablename__ = "users"
82
83     id = Column(Integer, primary_key=True, index=True, autoincrement=True)
84     name = Column(String(255), nullable=False)
85     email = Column(String(255), unique=True, nullable=False, index=True)
86     password_hash = Column(String(255), nullable=False)
87
88     sessions = relationship("Session", back_populates="user")
```

```
models.py M X
code > models.py > RestaurantInspection
30 class RestaurantInspection(Base):
50     restaurant_id = Column(
51         Integer,
52         ForeignKey("restaurants.id", ondelete="RESTRICT"),
53         nullable=False,
54     )
```

```
auth.py X
code > routers > auth.py > login
21 def login(
38     password_valid = bcrypt.checkpw(
39         credentials.password.encode("utf-8"),
40         user.password_hash.encode("utf-8"),
41     )
```

II. API Implementation

- Implementing CRUD endpoints for Restaurant and Restaurant Inspection entities.

Restaurant:

- POST /restaurants
- GET /restaurants
- GET /restaurants/{id}
- PUT /restaurants/{id}
- DELETE /restaurants/{id}

RestaurantInspection:

- POST /records
- GET /records
- GET /records/{id}
- PUT /records/{id}
- DELETE /records/{id}

- The Restaurant entity supports CRUD functionality. The list endpoint includes pagination using skip and limit. The delete endpoint also checks for associated RestaurantInspection records and returns a conflict if dependent records exist.

```

restaurant.py U
code > routers > restaurant.py > update_restaurant
15 @router.post(
16     "",
17     response_model=RestaurantResponse,
18     status_code=status.HTTP_201_CREATED,
19 )
20 def create_restaurant(
21     payload: RestaurantCreate,
22     db: Session = Depends(get_db),
23 ):
24     restaurant = Restaurant(
25         name=payload.name,
26         address=payload.address,
27         permit_code=payload.permit_code,
28     )
29     db.add(restaurant)
30
31     try:
32         db.commit()
33         db.refresh(restaurant)
34     except IntegrityError:
35         db.rollback()
36         raise HTTPException(
37             status_code=status.HTTP_409_CONFLICT,
38             detail="A restaurant with this permit code already exists.",
39         )
40
41     return restaurant
42
43 @router.get(
44     "",
45     response_model=List[RestaurantResponse],
46 )
47 def list_restaurants(
48     skip: int = Query(default=0, ge=0),
49     limit: int = Query(default=10, ge=1, le=100),
50     db: Session = Depends(get_db),
51 ):
52     restaurants = (
53         db.query(Restaurant)
54         .offset(skip)
55         .limit(limit)
56         .all()
57     )
58
59     return restaurants
60
61 @router.get(
62     "/{restaurant_id}",
63     response_model=RestaurantResponse,
64 )
65 def get_restaurant(
66     restaurant_id: int,
67     db: Session = Depends(get_db),
68 ):
69     restaurant = (
70         db.query(Restaurant)
71         .filter(Restaurant.id == restaurant_id)
72         .first()
73     )
74
75     if restaurant is None:
76         raise HTTPException(
77             status_code=status.HTTP_404_NOT_FOUND,
78             detail="Restaurant not found.",
79         )
80
81     return restaurant

```

```

restaurant.py U
code > routers > restaurant.py > ...
84 @router.put(
85     "/{restaurant_id}",
86     response_model=RestaurantResponse,
87 )
88 def update_restaurant(
89     restaurant_id: int,
90     payload: RestaurantUpdate,
91     db: Session = Depends(get_db),
92 ):
93     restaurant = (
94         db.query(Restaurant)
95         .filter(Restaurant.id == restaurant_id)
96         .first()
97     )
98
99     if restaurant is None:
100         raise HTTPException(
101             status_code=status.HTTP_404_NOT_FOUND,
102             detail="Restaurant not found.",
103         )
104
105     restaurant.name = payload.name
106     restaurant.address = payload.address
107     restaurant.permit_code = payload.permit_code
108
109     try:
110         db.commit()
111         db.refresh(restaurant)
112     except IntegrityError:
113         db.rollback()
114         raise HTTPException(
115             status_code=status.HTTP_409_CONFLICT,
116             detail="A restaurant with this permit code already exists.",
117         )
118
119     return restaurant
120
121 @router.delete(
122     "/{restaurant_id}",
123     status_code=status.HTTP_204_NO_CONTENT,
124 )
125 def delete_restaurant(
126     restaurant_id: int,
127     db: Session = Depends(get_db),
128 ):
129     restaurant = (
130         db.query(Restaurant)
131         .filter(Restaurant.id == restaurant_id)
132         .first()
133     )
134
135     if restaurant is None:
136         raise HTTPException(
137             status_code=status.HTTP_404_NOT_FOUND,
138             detail="Restaurant not found.",
139         )
140
141     associated_inspection = (
142         db.query(RestaurantInspection)
143         .filter(RestaurantInspection.restaurant_id == restaurant_id)
144         .first()
145     )
146
147     if associated_inspection is not None:
148         raise HTTPException(
149             status_code=status.HTTP_409_CONFLICT,
150             detail="Cannot delete restaurant with existing inspections.",
151         )
152
153     db.delete(restaurant)
154     db.commit()

```

- I extended the existing RestaurantInspection API to support CRUD functionality. The API can create inspection records, list records, retrieve a specific record by ID, update an existing record, and delete a record. The new fields inspection_code, violation_count, and restaurant_id are included, and the API validates that the referenced Restaurant exists before creating or updating an inspection.

```

code > routers > records.py > delete_record
19
20 @router.post(
21     "",
22     response_model=RestaurantInspectionResponse,
23     status_code=status.HTTP_201_CREATED,
24 )
25 def create_record(
26     record: RestaurantInspectionCreate,
27     db: DatabaseSession = Depends(get_db),
28     user: User = Depends(get_current_user),
29 ):
30     restaurant = (
31         db.query(Restaurant)
32         .filter(Restaurant.id == record.restaurant_id)
33         .first()
34     )
35
36     if restaurant is None:
37         raise HTTPException(
38             status_code=status.HTTP_404_NOT_FOUND,
39             detail="Restaurant not found.",
40         )
41
42     new_record = RestaurantInspection(
43         restaurant_name=record.restaurant_name,
44         restaurant_address=record.restaurant_address,
45         inspection_code=record.inspection_code,
46         violation_count=record.violation_count,
47         restaurant_id=record.restaurant_id,
48     )
49
50     db.add(new_record)
51
52     try:
53         db.commit()
54         db.refresh(new_record)
55     except IntegrityError:
56         db.rollback()
57         raise HTTPException(
58             status_code=status.HTTP_409_CONFLICT,
59             detail="An inspection with this inspection code already exists.",
60         )
61
62     return new_record
63
64
65 @router.get(
66     "",
67     response_model=list[RestaurantInspectionResponse],
68 )
69 def get_records(
70     skip: int = Query(default=0, ge=0),
71     limit: int = Query(default=10, ge=1, le=100),
72     db: DatabaseSession = Depends(get_db),
73     user: User = Depends(get_current_user),
74 ):
75     return (
76         db.query(RestaurantInspection)
77         .offset(skip)
78         .limit(limit)
79         .all()
80     )

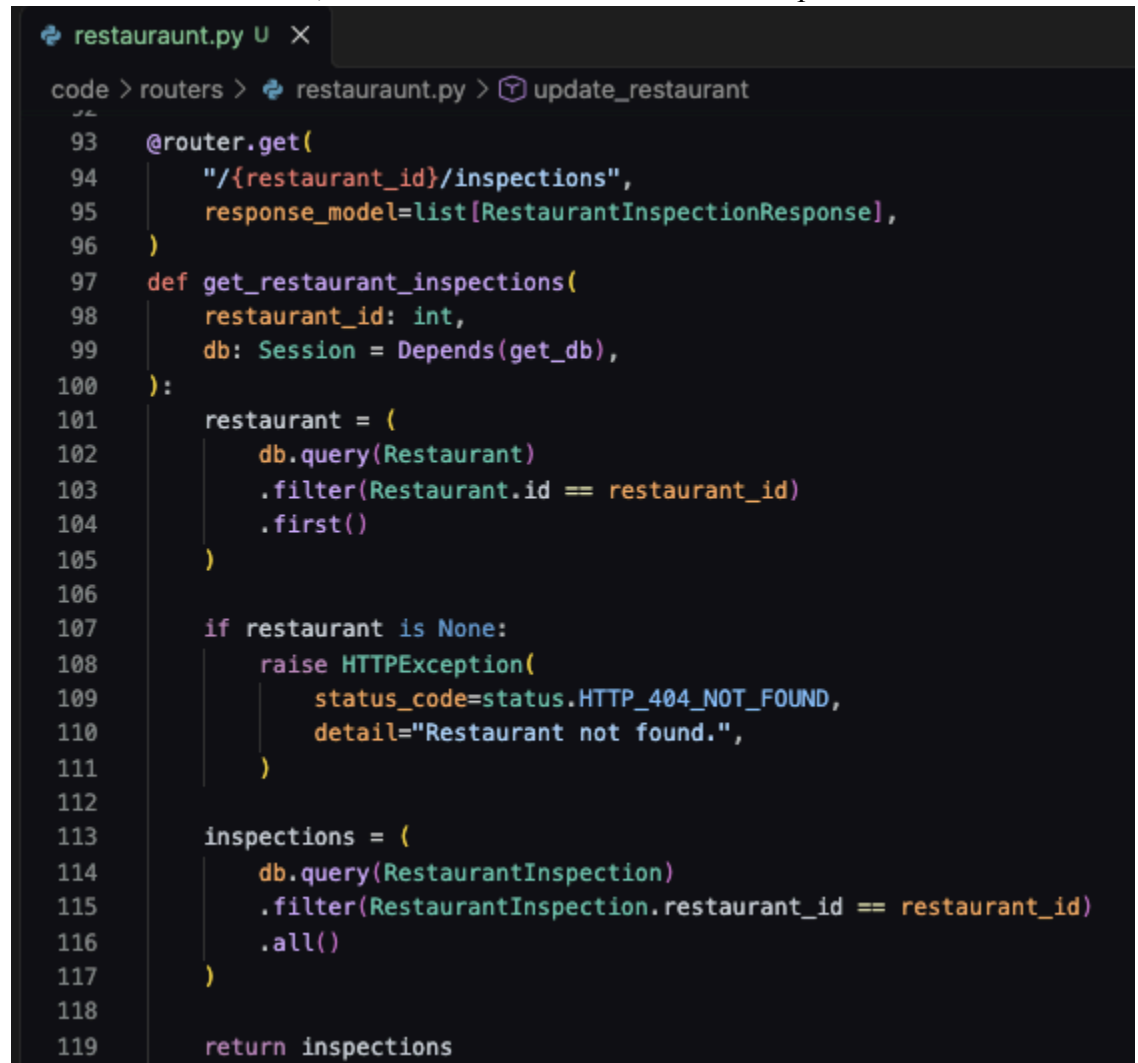
```

```

records.py M X
code > routers > records.py > ...
168 def get_record(
169     record_id: int,
170     db: DatabaseSession = Depends(get_db),
171     user: User = Depends(get_current_user),
172 ):
173     record = (
174         db.query(RestaurantInspection)
175         .filter(RestaurantInspection.id == record_id)
176         .first()
177     )
178
179     if record is None:
180         raise HTTPException(
181             status_code=status.HTTP_404_NOT_FOUND,
182             detail="Record not found.",
183         )
184
185     return record
186
187
188 @router.put(
189     "/{record_id}",
190     response_model=RestaurantInspectionResponse,
191 )
192 def update_record(
193     record_id: int,
194     updated_record: RestaurantInspectionUpdate,
195     db: DatabaseSession = Depends(get_db),
196     user: User = Depends(get_current_user),
197 ):
198     record = (
199         db.query(RestaurantInspection)
200         .filter(RestaurantInspection.id == record_id)
201         .first()
202     )
203
204     if record is None:
205         raise HTTPException(
206             status_code=status.HTTP_404_NOT_FOUND,
207             detail="Record not found.",
208         )
209
210     restaurant = (
211         db.query(Restaurant)
212         .filter(Restaurant.id == updated_record.restaurant_id)
213         .first()
214     )
215
216     if restaurant is None:
217         raise HTTPException(
218             status_code=status.HTTP_404_NOT_FOUND,
219             detail="Restaurant not found.",
220         )
221
222     record.restaurant_name = updated_record.restaurant_name
223     record.restaurant_address = updated_record.restaurant_address
224     record.inspection_code = updated_record.inspection_code
225     record.violation_count = updated_record.violation_count
226     record.restaurant_id = updated_record.restaurant_id
227
228     try:
229         db.commit()
230         db.refresh(record)
231     except IntegrityError:
232         db.rollback()
233         raise HTTPException(
234             status_code=status.HTTP_409_CONFLICT,
235             detail="An inspection with this inspection code already exists.",
236         )
237
238     return record
239
240
241 @router.delete(
242     "/{record_id}",
243     status_code=status.HTTP_204_NO_CONTENT,
244 )
245 def delete_record(
246     record_id: int,
247     db: DatabaseSession = Depends(get_db),
248     user: User = Depends(get_current_user),
249 ):
250     record = (
251         db.query(RestaurantInspection)
252         .filter(RestaurantInspection.id == record_id)
253         .first()
254     )
255
256     if record is None:
257         raise HTTPException(
258             status_code=status.HTTP_404_NOT_FOUND,
259             detail="Record not found.",
260         )
261
262     db.query(InspectionNote).filter(
263         InspectionNote.restaurant_inspection_id == record_id
264     ).delete()
265
266     db.delete(record)
267     db.commit()
268

```

- I added a relationship-query endpoint to retrieve all RestaurantInspection records associated with a specific Restaurant. The endpoint first verifies that the requested restaurant exists, then filters RestaurantInspection records by restaurant_id. If the restaurant does not exist, the API returns a 404 Not Found response.



```

code > routers > restaurant.py > update_restaurant
93 @router.get(
94    ("/{restaurant_id}/inspections",
95     response_model=list[RestaurantInspectionResponse],
96 )
97 def get_restaurant_inspections(
98     restaurant_id: int,
99     db: Session = Depends(get_db),
100 ):
101     restaurant = (
102         db.query(Restaurant)
103         .filter(Restaurant.id == restaurant_id)
104         .first()
105     )
106
107     if restaurant is None:
108         raise HTTPException(
109             status_code=status.HTTP_404_NOT_FOUND,
110             detail="Restaurant not found.",
111         )
112
113     inspections = (
114         db.query(RestaurantInspection)
115         .filter(RestaurantInspection.restaurant_id == restaurant_id)
116         .all()
117     )
118
119     return inspections

```

- I used Pydantic request and response schemas to validate API data for both Restaurant and RestaurantInspection. The unique fields are format-validated using regex patterns: permit_code must follow REST-<digits> and inspection_code must follow INSP-<digits>. Invalid request data is rejected by FastAPI/Pydantic with a 422 Unprocessable Entity response. The API also returns 404 Not Found for missing resources and 409 Conflict for constraint violations such as duplicate unique codes or attempting to delete a restaurant that still has associated inspection records.

```

schemas.py M X
code > schemas.py > RestaurantInspectionResponse
10
11 class RestaurantCreate(BaseModel):
12     name: str
13     address: str
14     permit_code: str = Field(pattern=r"^REST-\d+$")
15
16
17 class RestaurantUpdate(BaseModel):
18     name: str
19     address: str
20     permit_code: str = Field(pattern=r"^REST-\d+$")
21
22
23 class RestaurantResponse(BaseModel):
24     id: int
25     name: str
26     address: str
27     permit_code: str
28     created_at: datetime
29     updated_at: datetime
30
31     model_config = {
32         "from_attributes": True
33     }
34
35
36 class RestaurantInspectionCreate(BaseModel):
37     restaurant_name: str
38     restaurant_address: str
39     inspection_code: str = Field(pattern=r"^INSP-\d+$")
40     violation_count: int = Field(default=0, ge=0)
41     restaurant_id: int
42
43
44 class RestaurantInspectionUpdate(BaseModel):
45     restaurant_name: str
46     restaurant_address: str
47     inspection_code: str = Field(pattern=r"^INSP-\d+$")
48     violation_count: int = Field(ge=0)
49     restaurant_id: int

```

```

restaurant.py U X
code > routers > restaurant.py > delete_restaurant
126 def update_restaurant(
130
137     if restaurant is None:
138         raise HTTPException(
139             status_code=status.HTTP_404_NOT_FOUND,
140             detail="Restaurant not found.",
141         )
142
143     restaurant.name = payload.name
144     restaurant.address = payload.address
145     restaurant.permit_code = payload.permit_code
146
147     try:
148         db.commit()
149         db.refresh(restaurant)
150     except IntegrityError:
151         db.rollback()
152         raise HTTPException(
153             status_code=status.HTTP_409_CONFLICT,
154             detail="A restaurant with this permit code already exists.",
155         )
156
157     return restaurant

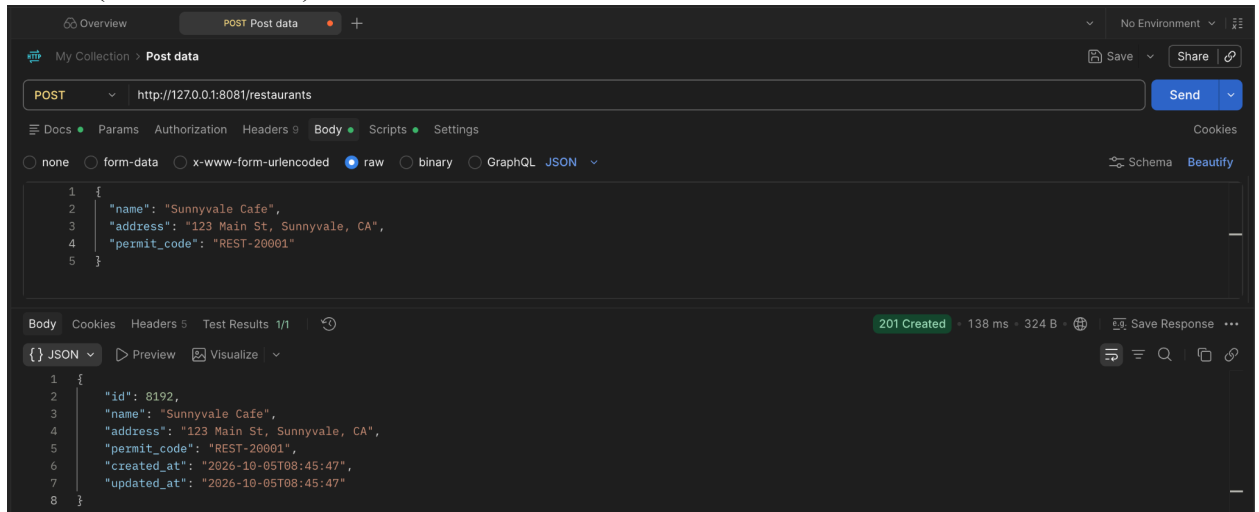
```

Song 9

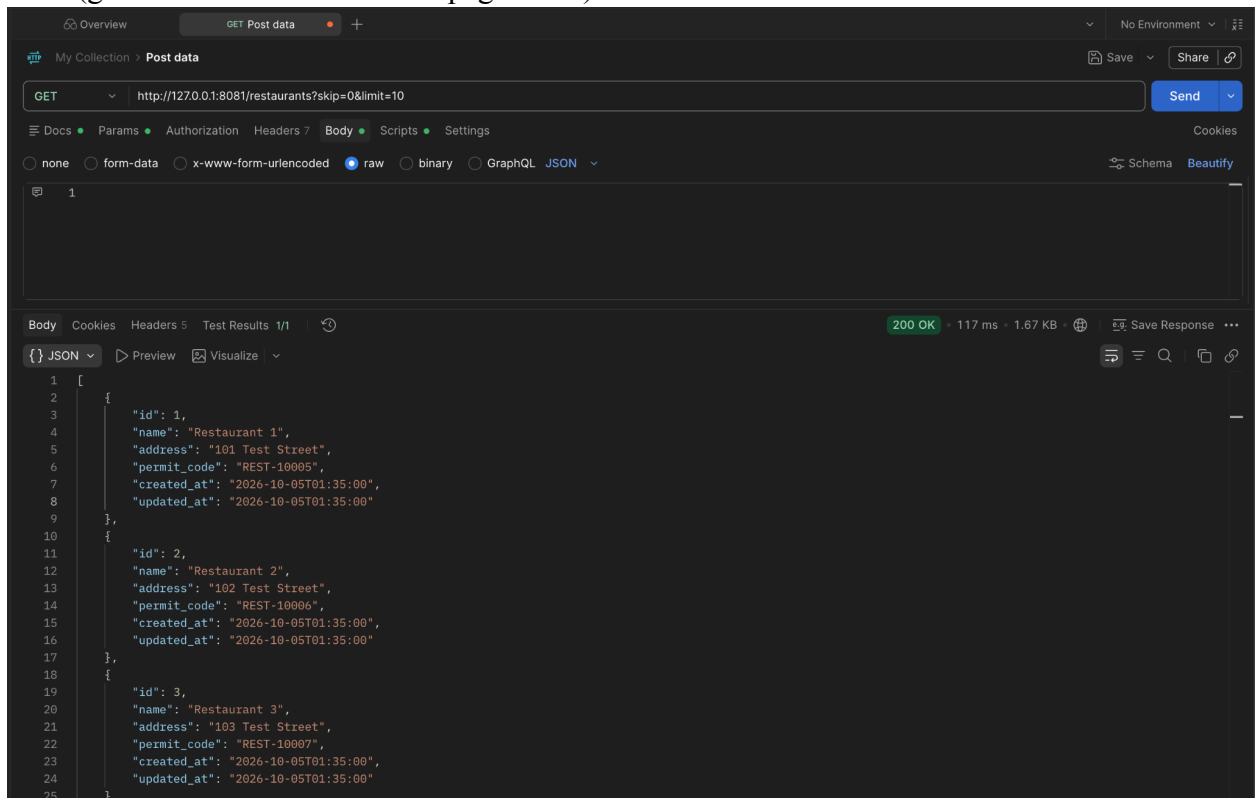
- Postman screenshots of each action tested using the API endpoints as well as a screenshot of my database.

Restaurant Entity:

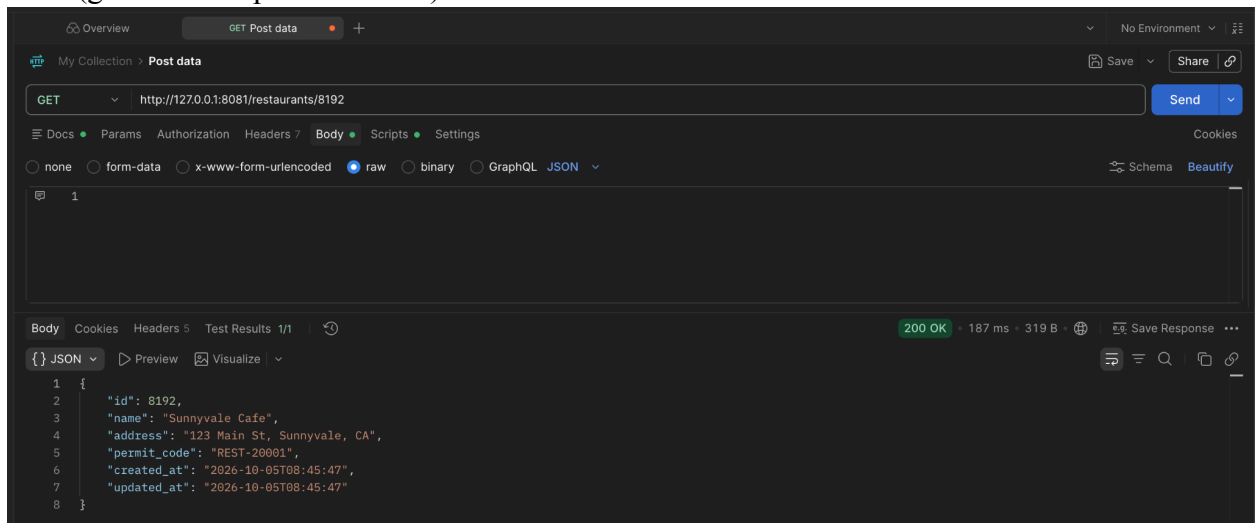
Create (add new record)



Read (get/retrieve all records with pagination)



Read (get/retrieve specific record)



Overview GET Post data + No Environment

My Collection > Post data Save Share

GET http://127.0.0.1:8081/restaurants/8192 Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

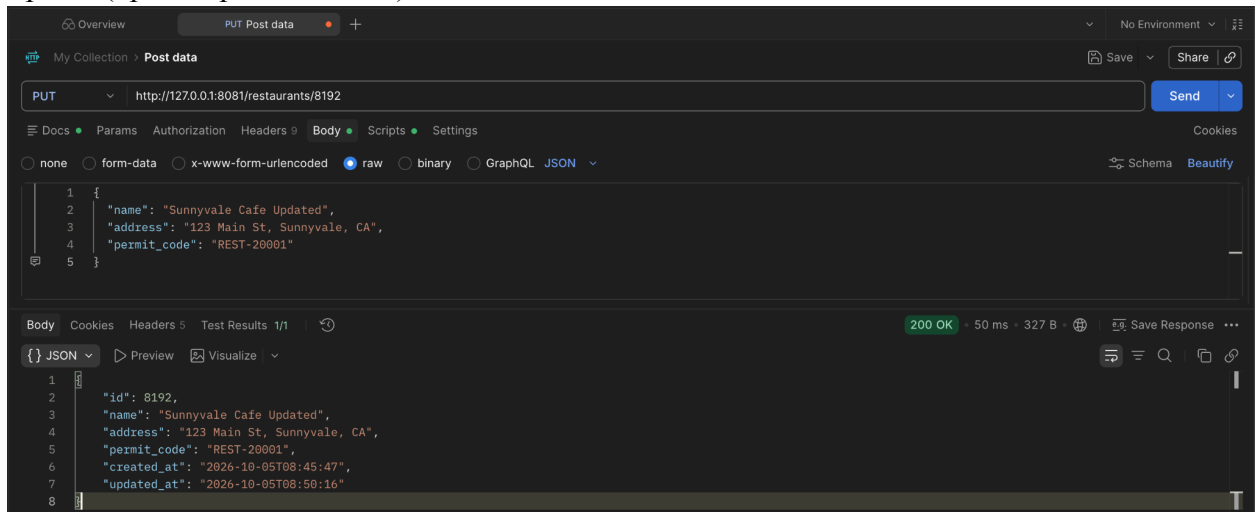
none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

1

Body Cookies Headers 5 Test Results 1/1 200 OK · 187 ms · 319 B Save Response

```
1 {
2   "id": 8192,
3   "name": "Sunnyvale Cafe",
4   "address": "123 Main St, Sunnyvale, CA",
5   "permit_code": "REST-20001",
6   "created_at": "2026-10-05T08:45:47",
7   "updated_at": "2026-10-05T08:45:47"
8 }
```

Update (update specific record)



Overview PUT Post data + No Environment

My Collection > Post data Save Share

PUT http://127.0.0.1:8081/restaurants/8192 Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

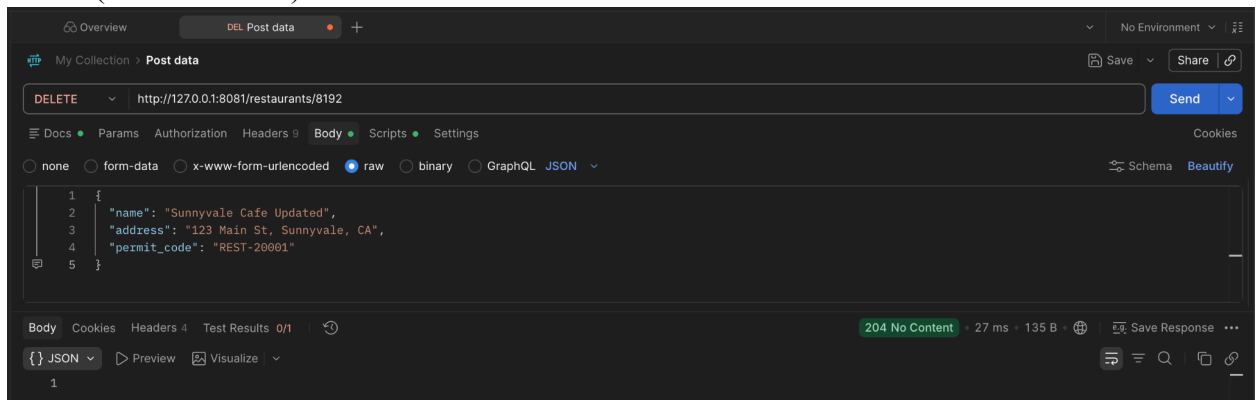
none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "name": "Sunnyvale Cafe Updated",
3   "address": "123 Main St, Sunnyvale, CA",
4   "permit_code": "REST-20001"
5 }
```

Body Cookies Headers 5 Test Results 1/1 200 OK · 50 ms · 327 B Save Response

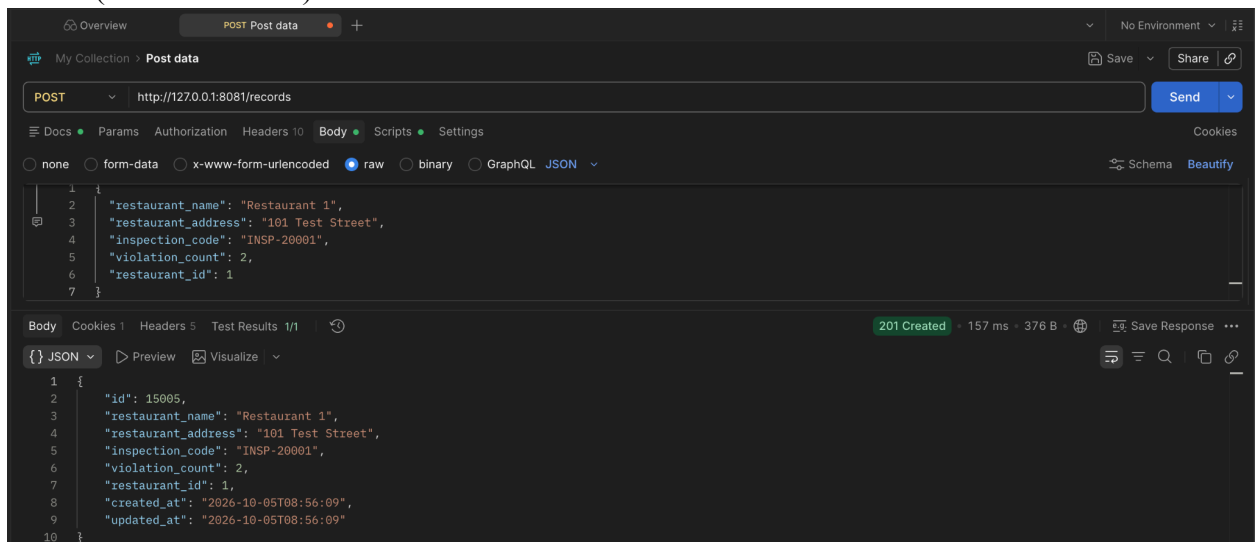
```
1 {
2   "id": 8192,
3   "name": "Sunnyvale Cafe Updated",
4   "address": "123 Main St, Sunnyvale, CA",
5   "permit_code": "REST-20001",
6   "created_at": "2026-10-05T08:45:47",
7   "updated_at": "2026-10-05T08:50:16"
8 }
```

Delete (remove record)

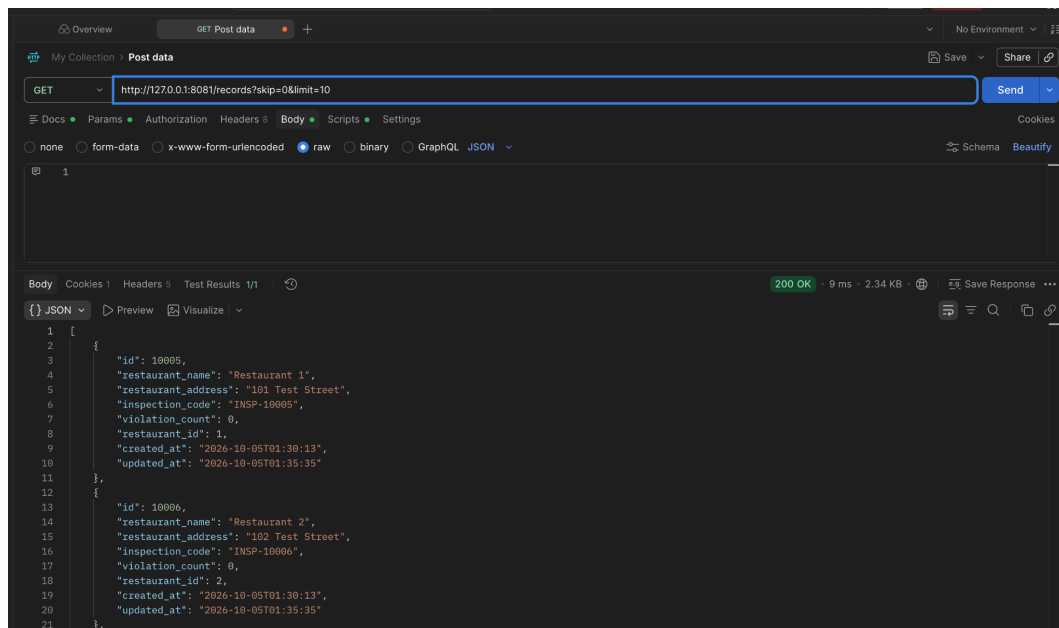


Restaurant Inspection entity:

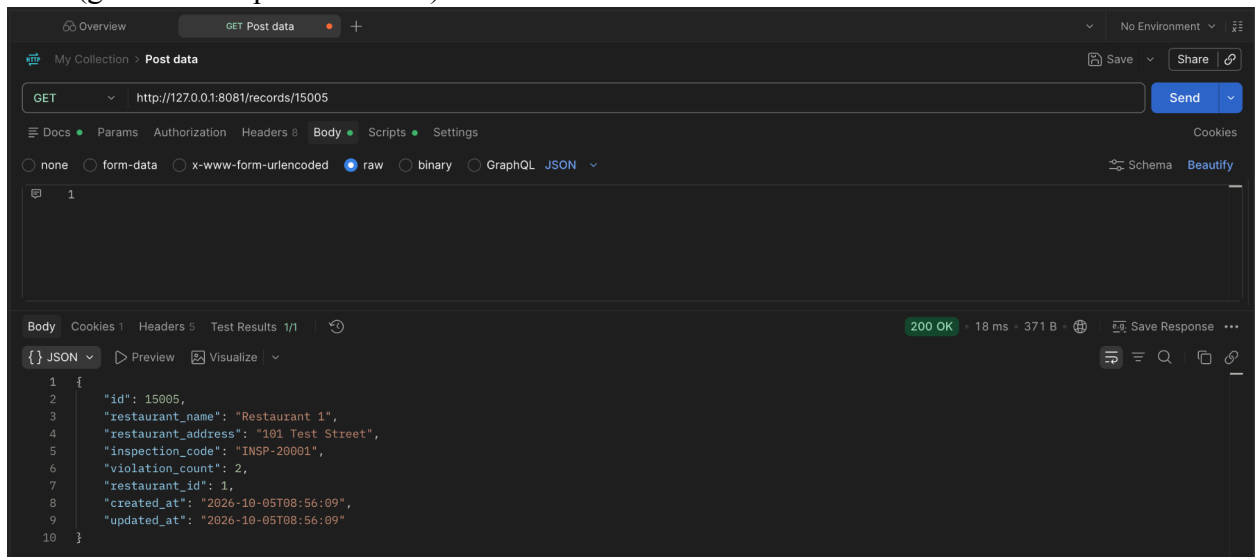
Create (add new record)



Read (get/retrieve all records with pagination)



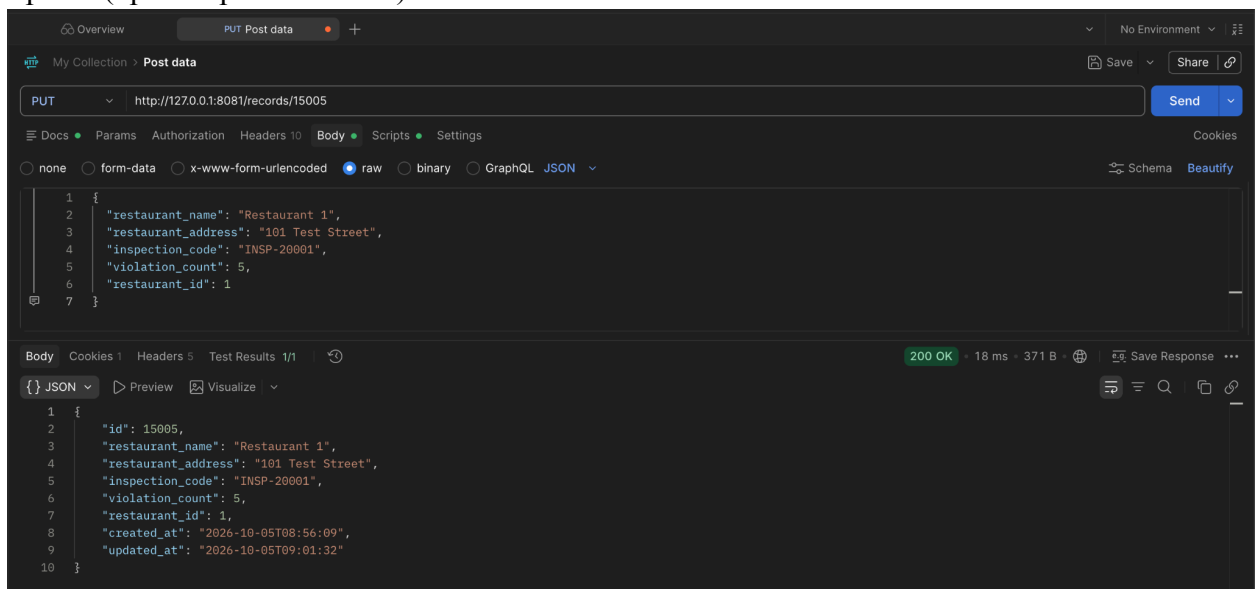
Read (get/retrieve specific record)



The screenshot shows a REST client interface with a GET request to `http://127.0.0.1:8081/records/15005`. The response is a 200 OK status with a JSON body. The JSON body contains the following data:

```
1 {
2   "id": 15005,
3   "restaurant_name": "Restaurant 1",
4   "restaurant_address": "101 Test Street",
5   "inspection_code": "INSP-20001",
6   "violation_count": 2,
7   "restaurant_id": 1,
8   "created_at": "2026-10-05T08:56:09",
9   "updated_at": "2026-10-05T08:56:09"
10 }
```

Update (update specific record)



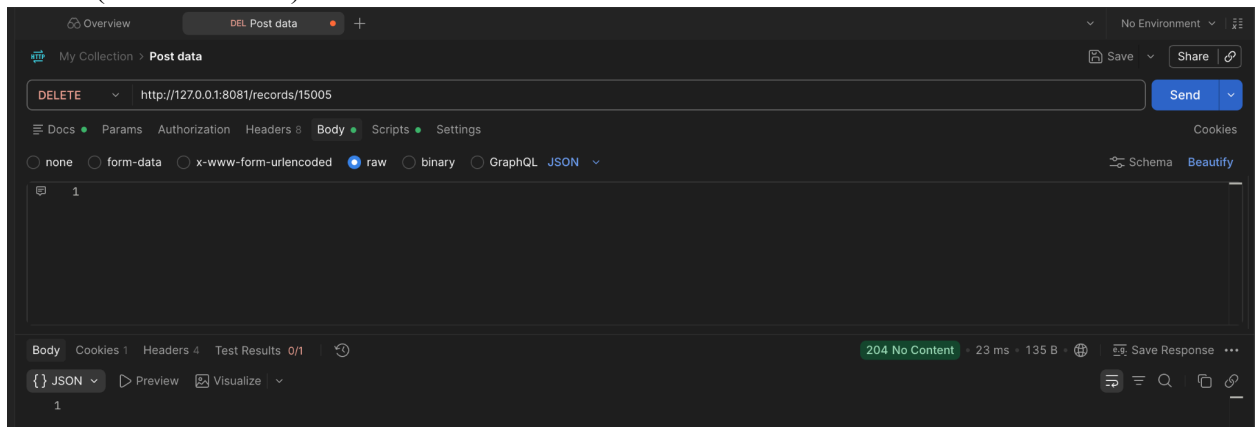
The screenshot shows a REST client interface with a PUT request to `http://127.0.0.1:8081/records/15005`. The request body is a JSON object with the following data:

```
1 {
2   "restaurant_name": "Restaurant 1",
3   "restaurant_address": "101 Test Street",
4   "inspection_code": "INSP-20001",
5   "violation_count": 5,
6   "restaurant_id": 1
7 }
```

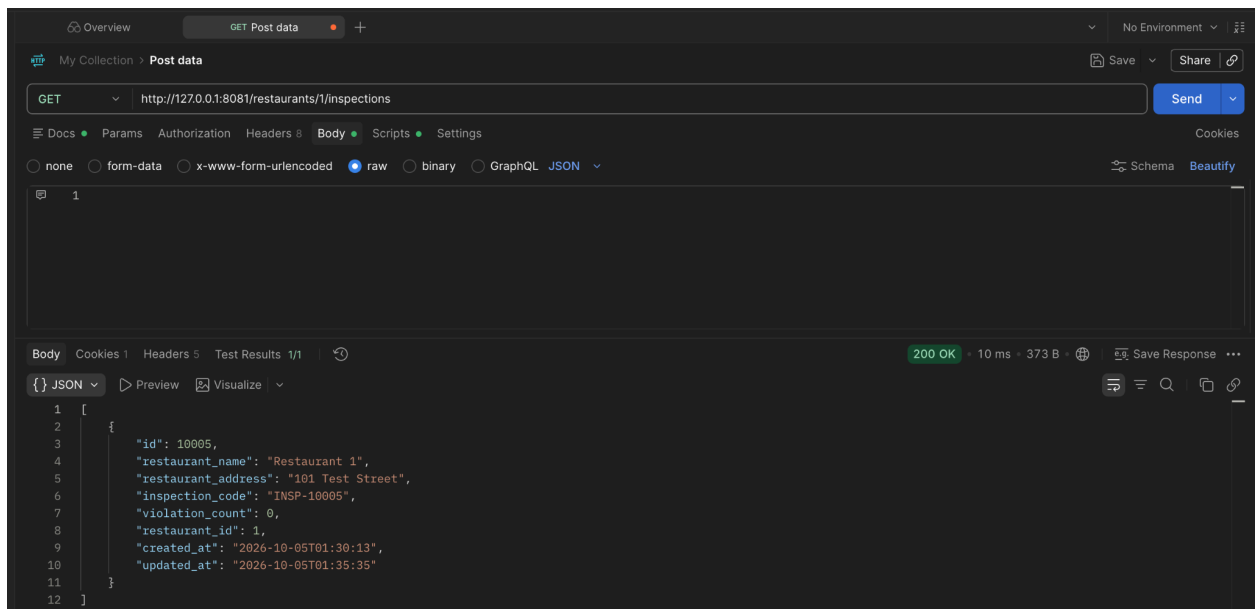
The response is a 200 OK status with a JSON body. The JSON body contains the following data:

```
1 {
2   "id": 15005,
3   "restaurant_name": "Restaurant 1",
4   "restaurant_address": "101 Test Street",
5   "inspection_code": "INSP-20001",
6   "violation_count": 5,
7   "restaurant_id": 1,
8   "created_at": "2026-10-05T08:56:09",
9   "updated_at": "2026-10-05T09:01:32"
10 }
```

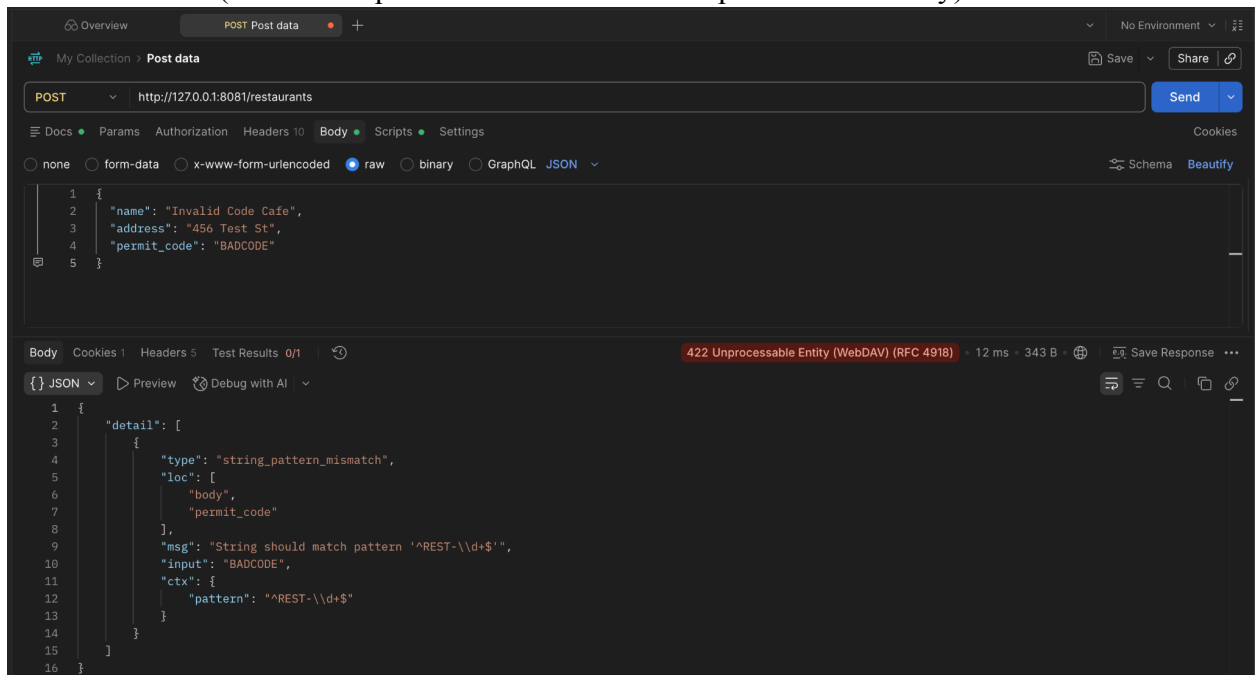
Delete (remove record)



Relationship Query (retrieve all inspections for a specific restaurant)



Validation Error (invalid unique-field format → 422 Unprocessable Entity)



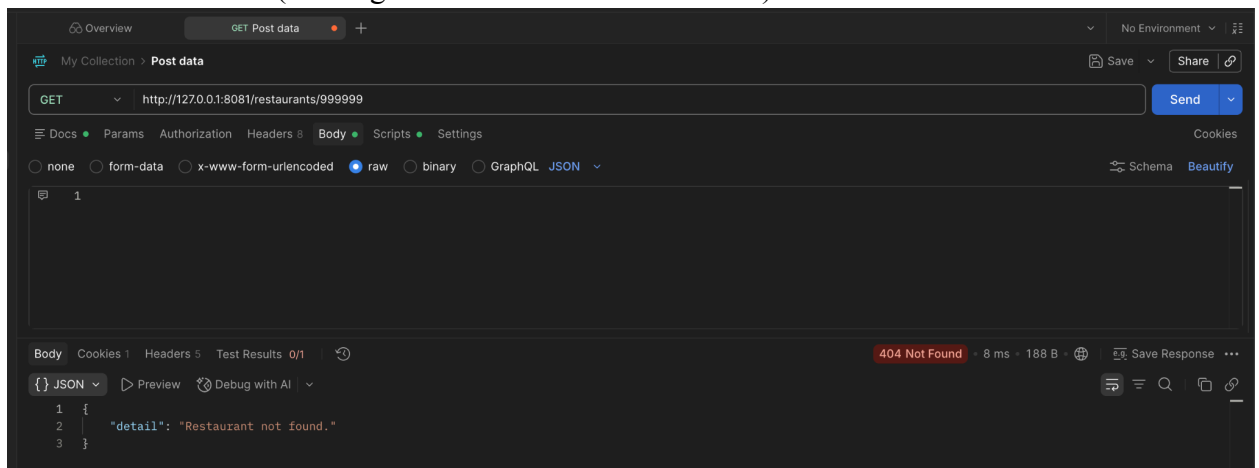
The screenshot shows a REST client interface with a POST request to `http://127.0.0.1:8081/restaurants`. The request body is raw JSON:

```
1 {
2   "name": "Invalid Code Cafe",
3   "address": "456 Test St",
4   "permit_code": "BADCODE"
5 }
```

The response status is **422 Unprocessable Entity (WebDAV) (RFC 4918)** with a 12 ms response time and 343 B of data. The response body is JSON:

```
1 {
2   "detail": [
3     {
4       "type": "string_pattern_mismatch",
5       "loc": [
6         "body",
7         "permit_code"
8       ],
9       "msg": "String should match pattern '^REST-\\d+$'",
10      "input": "BADCODE",
11      "ctx": {
12        "pattern": "REST-\\d+$"
13      }
14    }
15  ]
16 }
```

Resource Not Found (missing restaurant → 404 Not Found)



The screenshot shows a REST client interface with a GET request to `http://127.0.0.1:8081/restaurants/999999`. The response status is **404 Not Found** with an 8 ms response time and 188 B of data. The response body is JSON:

```
1 {
2   "detail": "Restaurant not found."
3 }
```

Constraint Violation (duplicate permit code → 409 Conflict)

The screenshot shows a REST client interface with a POST request to `http://127.0.0.1:8081/restaurants`. The request body is a JSON object:

```
{
  "name": "Duplicate Restaurant",
  "address": "999 Duplicate St",
  "permit_code": "REST-10005"
}
```

The response is a **409 Conflict** with a status of 6 ms and 216 B. The response body is:

```
{
  "detail": "A restaurant with this permit code already exists."
}
```

Database Verification — Restaurant and RestaurantInspection Tables

The screenshot shows a MySQL terminal session with the following queries and results:

```
mysql> USE s9981_rel;
Database changed
mysql> SELECT id, name, address, permit_code
  -> FROM restaurants
  -> LIMIT 10;
```

id	name	address	permit_code
1	Restaurant 1	101 Test Street	REST-10005
2	Restaurant 2	102 Test Street	REST-10006
3	Restaurant 3	103 Test Street	REST-10007
4	Restaurant 4	104 Test Street	REST-10008
5	Restaurant 5	105 Test Street	REST-10009
6	Restaurant 6	106 Test Street	REST-10010
7	Restaurant 7	107 Test Street	REST-10011
8	Restaurant 8	108 Test Street	REST-10012
9	Restaurant 9	109 Test Street	REST-10013
10	Restaurant 10	110 Test Street	REST-10014

10 rows in set (0.00 sec)

```
mysql> SELECT id, restaurant_name, inspection_code, violation_count, restaurant_id
  -> FROM restaurant_inspections
  -> LIMIT 10;
```

id	restaurant_name	inspection_code	violation_count	restaurant_id
10005	Restaurant 1	INSP-10005	0	1
10006	Restaurant 2	INSP-10006	0	2
10007	Restaurant 3	INSP-10007	0	3
10008	Restaurant 4	INSP-10008	0	4
10009	Restaurant 5	INSP-10009	0	5
10010	Restaurant 6	INSP-10010	0	6
10011	Restaurant 7	INSP-10011	0	7
10012	Restaurant 8	INSP-10012	0	8
10013	Restaurant 9	INSP-10013	0	9
10014	Restaurant 10	INSP-10014	0	10

10 rows in set (0.00 sec)

```
mysql>
```

III. Redux Client

1. Redux Setup

- I migrated the React data layer for the primary RestaurantInspection entity to Redux Toolkit. I created a Redux store and registered a records slice that manages inspection-record state. The slice stores the current records, loading state, and error state, and handles fetching, adding, updating, and deleting inspection records.



```
JS store.js U X
frontend > src > JS store.js > ...
1 import { configureStore } from "@reduxjs/toolkit";
2 import recordsReducer from "../features/recordsSlice";
3
4 export const store = configureStore({
5   reducer: {
6     records: recordsReducer,
7   },
8 });
```

I created a records slice to manage the state of the primary RestaurantInspection entity. The slice maintains the inspection records in items, along with loading and error state for asynchronous operations.

The records slice includes handlers for all required record operations. `fetchRecords` replaces the current Redux record list with data returned from the backend, `createRecord` appends a newly created inspection record, `updateRecordAsync` finds and replaces the matching record by ID, and `deleteRecordAsync` removes the deleted record from Redux state. The slice also handles pending and rejected cases by updating the loading and error state.

JS recordsSlive.js U X

frontend > src > features > JS recordsSlive.js > [🔍] default

```

65  const recordsSlice = createSlice({
66    name: "records",
67    initialState: {
68      items: [],
69      loading: false,
70      error: null,
71    },
72    reducers: {},
73    extraReducers: (builder) => {
74      builder
75        // Fetch
76        .addCase(fetchRecords.pending, (state) => {
77          state.loading = true;
78          state.error = null;
79        })
80        .addCase(fetchRecords.fulfilled, (state, action) => {
81          state.loading = false;
82          state.items = action.payload;
83        })
84        .addCase(fetchRecords.rejected, (state, action) => {
85          state.loading = false;
86          state.error = action.payload;
87        })
88
89        // Create
90        .addCase(createRecord.pending, (state) => {
91          state.loading = true;
92          state.error = null;
93        })
94        .addCase(createRecord.fulfilled, (state, action) => {
95          state.loading = false;
96          state.items.push(action.payload);
97        })
98        .addCase(createRecord.rejected, (state, action) => {
99          state.loading = false;
100          state.error = action.payload;
101        })
102
103        // Update
104        .addCase(updateRecordAsync.pending, (state) => {
105          state.loading = true;
106          state.error = null;
107        })
108        .addCase(updateRecordAsync.fulfilled, (state, action) => {
109          state.loading = false;
110
111          const index = state.items.findIndex(
112            (record) => record.id === action.payload.id
113          );
114
115          if (index !== -1) {
116            state.items[index] = action.payload;
117          }
118        })
119        .addCase(updateRecordAsync.rejected, (state, action) => {
120          state.loading = false;
121          state.error = action.payload;
122        })

```

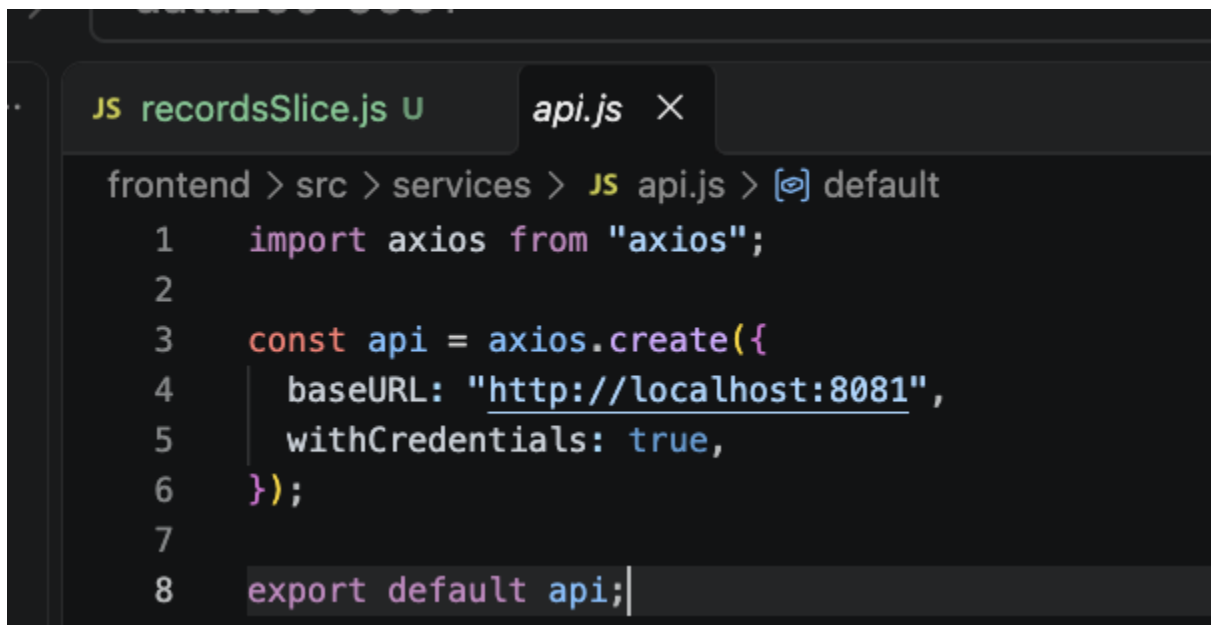
JS recordsSlive.js U X

frontend > src > features > JS recordsSlive.js > [🔍] default

```
65   const recordsSlice = createSlice({
73     extraReducers: (builder) => {
123
124       // Delete
125       .addCase(deleteRecordAsync.pending, (state) => {
126         state.loading = true;
127         state.error = null;
128       })
129       .addCase(deleteRecordAsync.fulfilled, (state, action) => {
130         state.loading = false;
131         state.items = state.items.filter(
132           (record) => record.id !== action.payload
133         );
134       })
135       .addCase(deleteRecordAsync.rejected, (state, action) => {
136         state.loading = false;
137         state.error = action.payload;
138       });
139     },
140   });
141
```

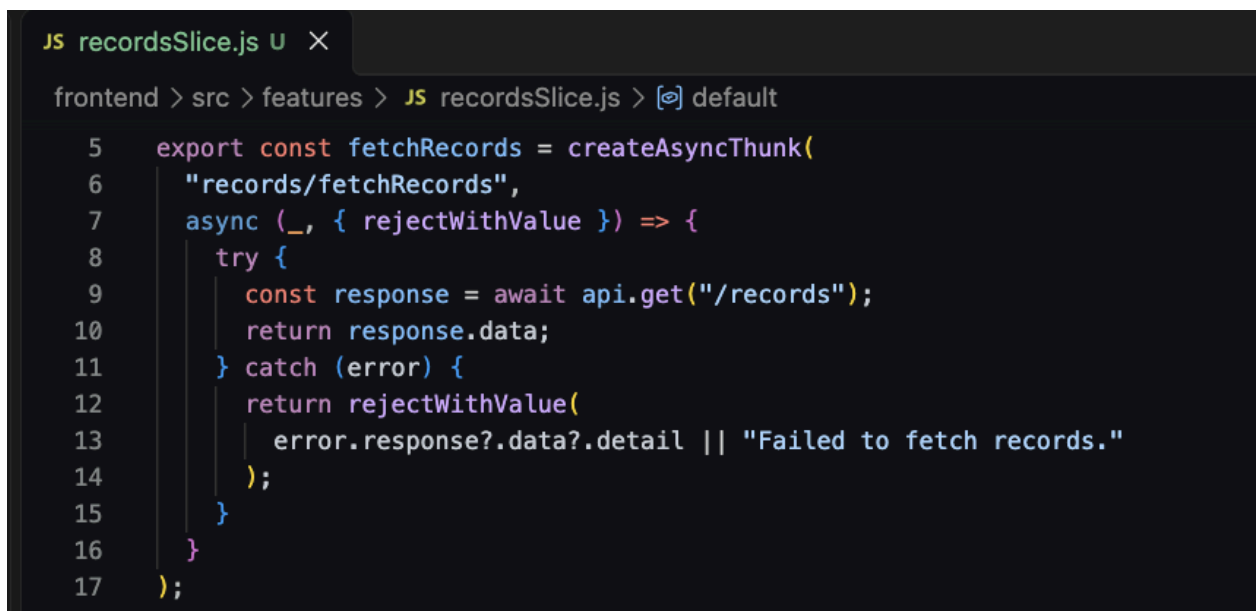
2. Async Thunks with Axios

- I implemented asynchronous Redux thunks using `createAsyncThunk` and the existing Axios API client. The thunks call the FastAPI backend for all required CRUD operations: `fetchRecords` sends a GET request to retrieve inspection records, `createRecord` sends a POST request to create a record, `updateRecordAsync` sends a PUT request to update a record by ID, and `deleteRecordAsync` sends a DELETE request to remove a record by ID. Each thunk catches API errors and returns an error message through `rejectWithValue`, while the slice handles pending, fulfilled, and rejected states to track loading, successful responses, and failures.



The screenshot shows a VS Code editor window with two tabs: `recordsSlice.js` and `api.js`. The `api.js` tab is active, showing the following code:

```
frontend > src > services > JS api.js > [🔍] default
1  import axios from "axios";
2
3  const api = axios.create({
4    baseURL: "http://localhost:8081",
5    withCredentials: true,
6  });
7
8  export default api;
```



The screenshot shows a VS Code editor window with two tabs: `recordsSlice.js` and `api.js`. The `recordsSlice.js` tab is active, showing the following code:

```
frontend > src > features > JS recordsSlice.js > [🔍] default
5  export const fetchRecords = createAsyncThunk(
6    "records/fetchRecords",
7    async (_, { rejectWithValue }) => {
8      try {
9        const response = await api.get("/records");
10       return response.data;
11     } catch (error) {
12       return rejectWithValue(
13         error.response?.data?.detail || "Failed to fetch records."
14       );
15     }
16   }
17 );
```

JS recordsSlice.js U X

frontend > src > features > JS recordsSlice.js > [🔍] default

```
19
20 export const createRecord = createAsyncThunk(
21   "records/createRecord",
22   async (recordData, { rejectWithValue }) => {
23     try {
24       const response = await api.post("/records", recordData);
25       return response.data;
26     } catch (error) {
27       return rejectWithValue(
28         error.response?.data?.detail || "Failed to create record."
29       );
30     }
31   }
32 );
```

JS recordsSlice.js U X

frontend > src > features > JS recordsSlice.js > [🔍] default

```
34
35 export const updateRecordAsync = createAsyncThunk(
36   "records/updateRecord",
37   async ({ id, recordData }, { rejectWithValue }) => {
38     try {
39       const response = await api.put(`/records/${id}`, recordData);
40       return response.data;
41     } catch (error) {
42       return rejectWithValue(
43         error.response?.data?.detail || "Failed to update record."
44       );
45     }
46   }
47 );
48
49
50 export const deleteRecordAsync = createAsyncThunk(
51   "records/deleteRecord",
52   async (id, { rejectWithValue }) => {
53     try {
54       await api.delete(`/records/${id}`);
55       return id;
56     } catch (error) {
57       return rejectWithValue(
58         error.response?.data?.detail || "Failed to delete record."
59       );
60     }
61   }
62 );
```

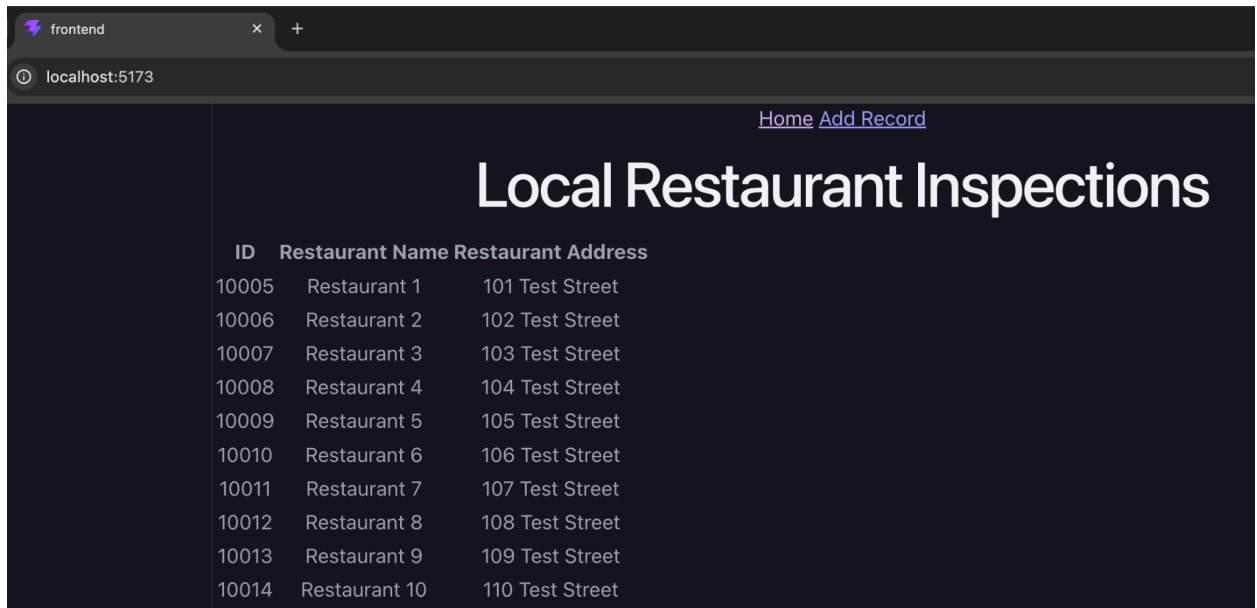
3. Home Screen

- I updated the Home screen to use Redux as the source of inspection-record data instead of local React state. The component dispatches `fetchRecords()` when an authenticated user opens the page and reads the record list, loading state, and error state from `state.records` using `useSelector`. The table renders directly from the Redux items array. Because the create, update, and delete thunks modify this same Redux state, the displayed list automatically reflects successful CRUD changes without requiring a separate local copy of the records.

```

Home.jsx M X
frontend > src > pages > Home.jsx > default
4
5 function Home({ user }) {
6   const dispatch = useDispatch();
7
8   const records = useSelector((state) => state.records.items);
9   const loading = useSelector((state) => state.records.loading);
10  const error = useSelector((state) => state.records.error);
11
12  useEffect(() => {
13    if (user) {
14      dispatch(fetchRecords());
15    }
16  }, [dispatch, user]);
17
18  if (!user) {
19    return (
20      <div>
21        <h1>Local Restaurant Inspections</h1>
22        <p>Login required</p>
23      </div>
24    );
25  }
26
27  return (
28    <div>
29      <h1>Local Restaurant Inspections</h1>
30
31      {loading && <p>Loading records...</p>}
32
33      {error && <p>{error}</p>}
34
35      {!loading && records.length === 0 ? (
36        <p>No records found.</p>
37      ) : (
38        <table>
39          <thead>
40            <tr>
41              <th>ID</th>
42              <th>Restaurant Name</th>
43              <th>Restaurant Address</th>
44            </tr>
45          </thead>
46
47          <tbody>
48            {records.map((record) => (
49              <tr key={record.id}>
50                <td>{record.id}</td>
51                <td>{record.restaurant_name}</td>
52                <td>{record.restaurant_address}</td>
53              </tr>
54            ))}
55          </tbody>
56        </table>
57      )}
58    </div>
59  );
60 }
61
62 export default Home;

```



ID	Restaurant Name	Restaurant Address
10005	Restaurant 1	101 Test Street
10006	Restaurant 2	102 Test Street
10007	Restaurant 3	103 Test Street
10008	Restaurant 4	104 Test Street
10009	Restaurant 5	105 Test Street
10010	Restaurant 6	106 Test Street
10011	Restaurant 7	107 Test Street
10012	Restaurant 8	108 Test Street
10013	Restaurant 9	109 Test Street
10014	Restaurant 10	110 Test Street

4. Create/Update Screens

- I implemented separate Create and Update forms for RestaurantInspection records. The Create form collects all required inspection fields and dispatches the createRecord Redux thunk, which sends a POST request to the FastAPI backend and appends the returned record to Redux state. The Update form allows an existing record to be selected by ID, collects the updated field values, and dispatches updateRecordAsync, which sends a PUT request to /records/{id}. After a successful create or update, the application navigates back to the Home screen, where the updated Redux state is rendered automatically.

```
CreateRecord.jsx M X
frontend > src > pages > CreateRecord.jsx > default
6  function CreateRecord({ user }) {
23  const handleSubmit = async (event) => {
26      const resultAction = await dispatch(
27          createRecord({
28              restaurant_name: restaurantName,
29              restaurant_address: restaurantAddress,
30              inspection_code: inspectionCode,
31              violation_count: Number(violationCount),
32              restaurant_id: Number(restaurantId),
33          })
34      );
35
36      if (createRecord.fulfilled.match(resultAction)) {
37          navigate("/");
38      }
39  };
```

frontend

localhost:5173/create

[Home](#) [Add Record](#)

Add Restaurant Inspection

Restaurant Name

Restaurant Address

Inspection Code

Violation Count

Restaurant ID

frontend x +

localhost:5173

[Home](#) [Add Record](#)

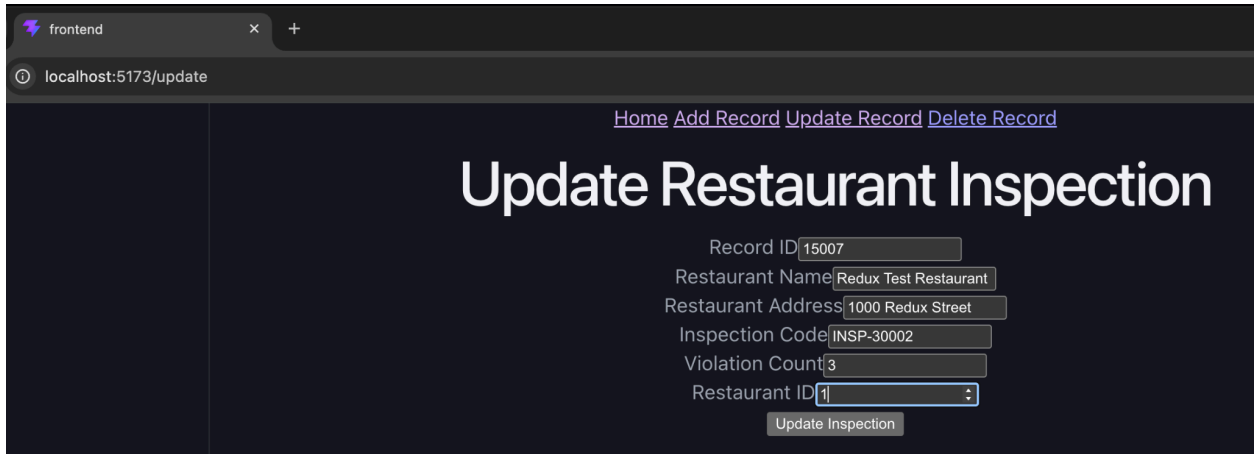
Local Restaurant Inspections

ID	Restaurant Name	Restaurant Address
10005	Restaurant 1	101 Test Street
10006	Restaurant 2	102 Test Street
10007	Restaurant 3	103 Test Street
10008	Restaurant 4	104 Test Street
10009	Restaurant 5	105 Test Street
10010	Restaurant 6	106 Test Street
10011	Restaurant 7	107 Test Street
10012	Restaurant 8	108 Test Street
10013	Restaurant 9	109 Test Street
10014	Restaurant 10	110 Test Street
15007	Redux Test Restaurant 2	1000 Redux Street
15008	Redux Test Restaurant 3	1234 Redux Street

UpdateRecord.jsx M X

frontend > src > pages > UpdateRecord.jsx > default

```
6 function UpdateRecord({ user }) {
24   const handleSubmit = async (event) => {
26
27     const resultAction = await dispatch(
28       updateRecordAsync({
29         id: Number(recordId),
30         recordData: {
31           restaurant_name: restaurantName,
32           restaurant_address: restaurantAddress,
33           inspection_code: inspectionCode,
34           violation_count: Number(violationCount),
35           restaurant_id: Number(restaurantId),
36         },
37       })
38     );
39
40     if (updateRecordAsync.fulfilled.match(resultAction)) {
41       navigate("/");
42     }
43   };
}
```

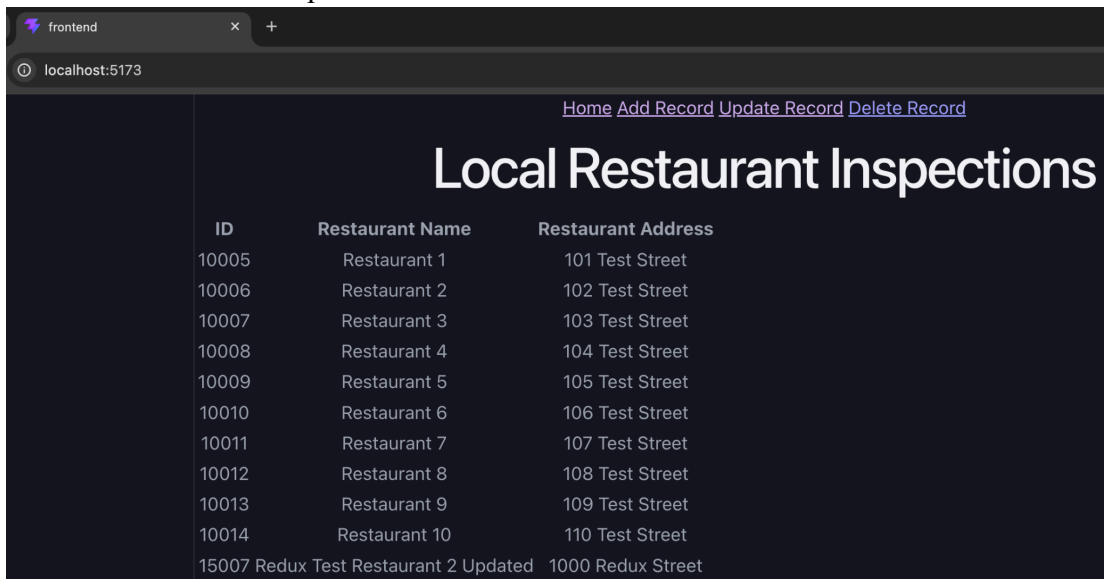


The screenshot shows a web browser window with the address bar at `localhost:5173/update`. The page has a dark theme and a navigation bar with links: [Home](#), [Add Record](#), [Update Record](#), and [Delete Record](#). The main heading is "Update Restaurant Inspection". Below the heading is a form with the following fields:

- Record ID:
- Restaurant Name:
- Restaurant Address:
- Inspection Code:
- Violation Count:
- Restaurant ID:

At the bottom of the form is a button labeled "Update Inspection".

*The Restaurant Name field is visually truncated in the form because of the input width; the full submitted value was Redux Test Restaurant 2 Updated, as confirmed on the Home screen after the update.

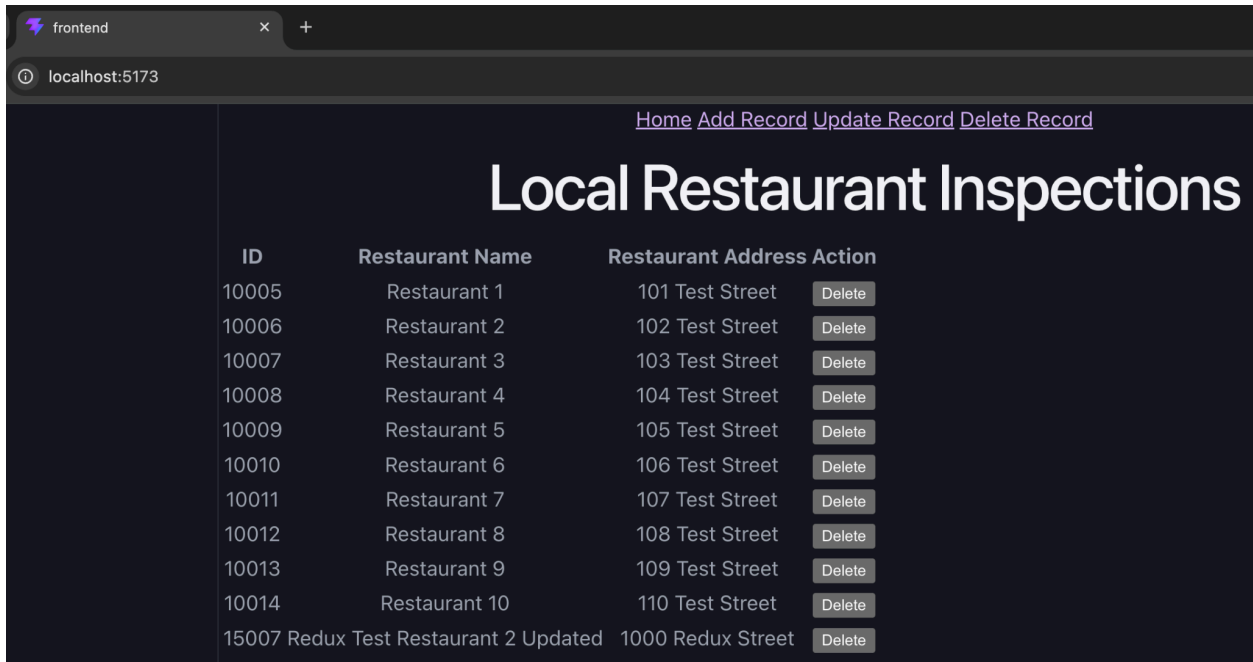


The screenshot shows a web browser window with the address bar at `localhost:5173`. The page has a dark theme and a navigation bar with links: [Home](#), [Add Record](#), [Update Record](#), and [Delete Record](#). The main heading is "Local Restaurant Inspections". Below the heading is a table with the following data:

ID	Restaurant Name	Restaurant Address
10005	Restaurant 1	101 Test Street
10006	Restaurant 2	102 Test Street
10007	Restaurant 3	103 Test Street
10008	Restaurant 4	104 Test Street
10009	Restaurant 5	105 Test Street
10010	Restaurant 6	106 Test Street
10011	Restaurant 7	107 Test Street
10012	Restaurant 8	108 Test Street
10013	Restaurant 9	109 Test Street
10014	Restaurant 10	110 Test Street
15007	Redux Test Restaurant 2 Updated	1000 Redux Street

5. Delete Record

- I added a Delete button next to each inspection record on the Home screen. When the button is clicked, the component dispatches `deleteRecordAsync` with that record's ID. After the backend successfully deletes the record, the Redux fulfilled handler removes the matching record from `state.records.items`, causing the Home list to update automatically without maintaining a separate local copy of the data.



The screenshot shows a web application running on `localhost:5173`. The page has a dark theme and a navigation bar at the top with links: [Home](#), [Add Record](#), [Update Record](#), and [Delete Record](#). The main heading is

Local Restaurant Inspections

. Below the heading is a table with 15 rows of restaurant inspection data. Each row has columns for ID, Restaurant Name, Restaurant Address, and an Action button labeled 'Delete'.

ID	Restaurant Name	Restaurant Address	Action
10005	Restaurant 1	101 Test Street	Delete
10006	Restaurant 2	102 Test Street	Delete
10007	Restaurant 3	103 Test Street	Delete
10008	Restaurant 4	104 Test Street	Delete
10009	Restaurant 5	105 Test Street	Delete
10010	Restaurant 6	106 Test Street	Delete
10011	Restaurant 7	107 Test Street	Delete
10012	Restaurant 8	108 Test Street	Delete
10013	Restaurant 9	109 Test Street	Delete
10014	Restaurant 10	110 Test Street	Delete
15007	Redux Test Restaurant 2 Updated	1000 Redux Street	Delete

- I tested the Delete button on record ID 10015. After clicking Delete, the Redux `deleteRecordAsync` thunk sent the DELETE request to the FastAPI backend. When the request succeeded, Redux removed record 10015 from `state.records.items`, and the Home screen updated automatically. The following screenshot confirms that record 10015 is no longer displayed.

The image shows three parts: two code editor snippets and a browser screenshot.

Code Editor 1 (Home.jsx): Shows the `handleDelete` function using `async` and `await` to dispatch `deleteRecordAsync(id)`.

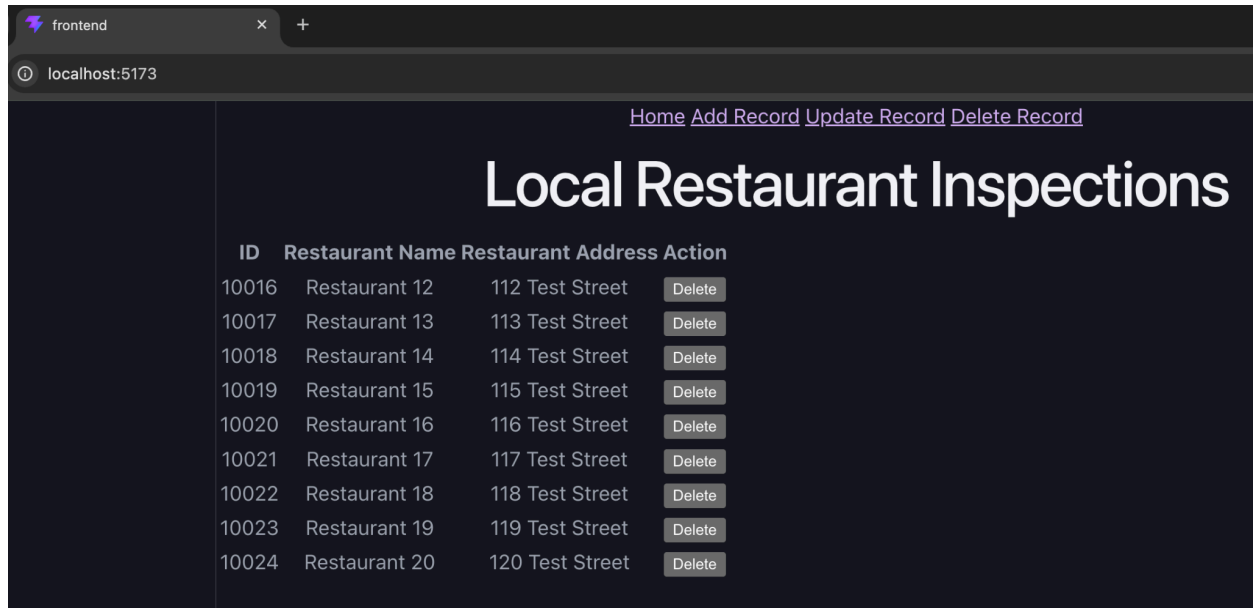
```
function Home({ user }) {  
  21   const handleDelete = async (id) => {  
  22     await dispatch(deleteRecordAsync(id));  
  23   };  
}
```

Code Editor 2 (Home.jsx): Shows the `records.map()` callback in the `Home` component, rendering a `button` with `onClick={handleDelete(record.id)}` and `disabled={loading}`.

```
function Home({ user }) {  
  56   {records.map((record) => (  
  63     <button  
  64       type="button"  
  65       onClick={() => handleDelete(record.id)}  
  66       disabled={loading}  
  67       >  
  68         Delete  
    </button>  
  )  
)}  
}
```

Browser Screenshot: Shows the application running on `localhost:5173`. The page title is "Local Restaurant Inspections". The navigation bar includes links: [Home](#), [Add Record](#), [Update Record](#), and [Delete Record](#). The main content is a table with 10 columns: ID, Restaurant Name, Restaurant Address, and Action. The table lists records from ID 10015 to 10024. Each record has a "Delete" button in the Action column. Record 10015 is no longer present in the table.

ID	Restaurant Name	Restaurant Address	Action
10015	Restaurant 11	111 Test Street	Delete
10016	Restaurant 12	112 Test Street	Delete
10017	Restaurant 13	113 Test Street	Delete
10018	Restaurant 14	114 Test Street	Delete
10019	Restaurant 15	115 Test Street	Delete
10020	Restaurant 16	116 Test Street	Delete
10021	Restaurant 17	117 Test Street	Delete
10022	Restaurant 18	118 Test Street	Delete
10023	Restaurant 19	119 Test Street	Delete
10024	Restaurant 20	120 Test Street	Delete



Part 2 - MCP Tool Servers

A. TheMealDB Tutorial Server

6. I implemented the `search_meals_by_name` MCP tool using FastMCP. The tool calls TheMealDB `search-by-name` endpoint with the provided query and validates that the limit is between 1 and 25. It returns up to the requested number of meals with the meal ID, name, cuisine area, category, and thumbnail URL. I tested the tool in MCP Inspector using the query `Arrabiata` with a limit of 5 and confirmed that the returned output follows the required schema.

```
meals_server.py 1, U X
code > meals_server.py > ...
13 @mcp.tool()
14 def search_meals_by_name(query: str, limit: int = 5):
15     if limit < 1 or limit > 25:
16         raise ValueError("limit must be between 1 and 25")
17
18     try:
19         response = httpx.get(
20             f"{BASE_URL}/search.php",
21             params={"s": query},
22             timeout=10.0,
23         )
24         response.raise_for_status()
25         data = response.json()
26     except (httpx.HTTPError, ValueError) as error:
27         raise RuntimeError(f"TheMealDB request failed: {error}") from error
28
29     meals = data.get("meals")
30
31     if meals is None:
32         return {
33             "message": "no matches",
34             "results": [],
35         }
36
37     results = []
38
39     for meal in meals[:limit]:
40         results.append(
41             {
42                 "id": meal["idMeal"],
43                 "name": meal["strMeal"],
44                 "area": meal["strArea"],
45                 "category": meal["strCategory"],
46                 "thumb": meal["strMealThumb"],
47             }
48         )
49
50     return results
```

127.0.0.1:6274/?MCP_INSPECTOR_API_TOKEN=c1f147579bf3d550d125bdb7773b93e911c733aee9824ec3d5bd9a966b6dfa09

meals Servers Tools Prompts Resources

Tools

Search tools...

☒ Paginated

search_meals_by_name

search_meals_by_name

Edit as JSON ☒

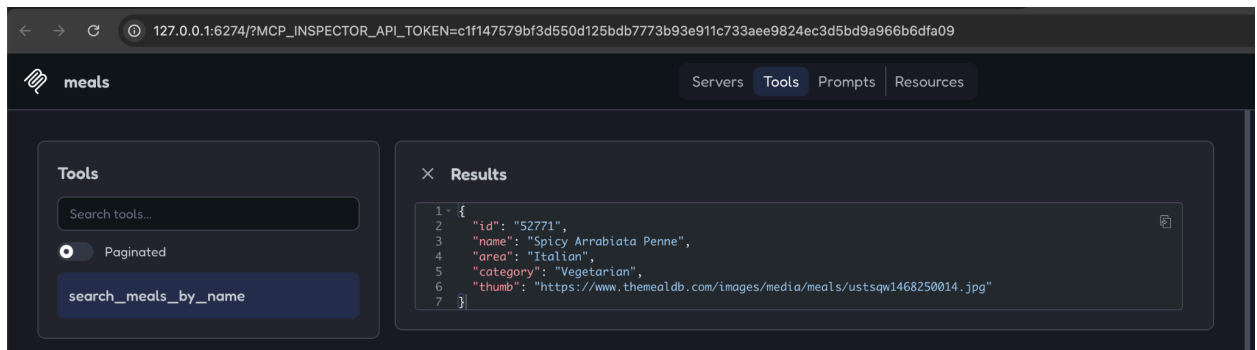
Query *

Arrabiata

Limit

5

Execute Tool



- I implemented the `meals_by_ingredient` MCP tool using FastMCP. The tool calls TheMealDB filter-by-main-ingredient endpoint with the provided ingredient and validates that limit is between 1 and 25. It returns up to the requested number of meals, with each result containing the meal ID, name, and thumbnail URL. I tested the tool in MCP Inspector using chicken with a limit of 12 and confirmed that the returned results follow the required schema.

```
meals_server.py 1, U X
code > meals_server.py > ...
53 @mcp.tool()
54 def meals_by_ingredient(ingredient: str, limit: int = 12):
55     if limit < 1 or limit > 25:
56         raise ValueError("limit must be between 1 and 25")
57
58     try:
59         response = httpx.get(
60             f"{BASE_URL}/filter.php",
61             params={"i": ingredient},
62             timeout=10.0,
63         )
64         response.raise_for_status()
65         data = response.json()
66     except (httpx.HTTPError, ValueError) as error:
67         raise RuntimeError(f"TheMealDB request failed: {error}") from error
68
69     meals = data.get("meals")
70
71     if meals is None:
72         return {
73             "message": "no matches",
74             "results": [],
75         }
76
77     results = []
78
79     for meal in meals[:limit]:
80         results.append(
81             {
82                 "id": meal["idMeal"],
83                 "name": meal["strMeal"],
84                 "thumb": meal["strMealThumb"],
85             }
86         )
87
88     return results
```

The screenshot displays the MCP Inspector web interface, which is used for managing and interacting with various tools. The interface is divided into two main sections: a left sidebar for tool management and a right pane for tool configuration and results.

Top Section: The browser address bar shows the URL `127.0.0.1:6274/?MCP_INSPECTOR_API_TOKEN=c1f147579bf3d550d125bdb7773b93e911c733aee9824ec3d5bd9a966b6dfa09`. The page title is "meals". The navigation bar includes "Servers", "Tools", "Prompts", and "Resources".

Left Sidebar (Tools): This section contains a search bar labeled "Search tools...". Below it, there is a toggle switch for "Paginated" (currently turned on). A list of tools is shown, including "search_meals_by_name" and "meals_by_ingredient", with the latter being selected.

Right Pane (meals_by_ingredient): This section is titled "meals_by_ingredient". It features an "Ingredient" input field with the value "chicken" and a "Limit" input field with the value "12". An "Execute Tool" button is located at the bottom. The "Edit as JSON" toggle is turned on.

Results Section: Below the "Execute Tool" button, the results of the tool execution are displayed. The results are a list of JSON objects, each representing a meal. The first five results are shown, each with a line number (1-5) and a copy icon.

```
1 {
2   "id": "52940",
3   "name": "Brown Stew Chicken",
4   "thumb": "https://www.themealdb.com/images/media/meals/sypxpx1515365095.jpg"
5 }
```

```
1 {
2   "id": "53161",
3   "name": "Chicken & chorizo rice pot",
4   "thumb": "https://www.themealdb.com/images/media/meals/fk80jp1763280767.jpg"
5 }
```

```
1 {
2   "id": "52846",
3   "name": "Chicken & mushroom Hotpot",
4   "thumb": "https://www.themealdb.com/images/media/meals/uuspp1511297945.jpg"
5 }
```

```
1 {
2   "id": "52796",
3   "name": "Chicken Alfredo Primavera",
4   "thumb": "https://www.themealdb.com/images/media/meals/syqypv1486981727.jpg"
5 }
```

```
1 {
2   "id": "52934",
3   "name": "Chicken Basquaise",
4   "thumb": "https://www.themealdb.com/images/media/meals/wruvqv1511880994.jpg"
5 }
```

```
1 {
2   "id": "52956",
3   "name": "Chicken Congee",
4   "thumb": "https://www.themealdb.com/images/media/meals/1529446352.jpg"
5 }
```

```
1 {
2   "id": "52795",
3   "name": "Chicken Handi",
4   "thumb": "https://www.themealdb.com/images/media/meals/wyxwsp1486979827.jpg"
5 }
```

```
1 {
2   "id": "52831",
3   "name": "Chicken Karaage",
4   "thumb": "https://www.themealdb.com/images/media/meals/tyyws1505930373.jpg"
5 }
```

8. I implemented the `random_meal` MCP tool using FastMCP. The tool calls TheMealDB `/random.php` endpoint to retrieve one random meal. The returned meal is passed through the shared `format_meal_details` helper so the output follows the required detailed schema, including the meal ID, name, category, cuisine area, instructions, image, source URL, YouTube URL, and ingredient/measure pairs. I tested the tool in MCP Inspector and confirmed that it returns a single complete meal object in the expected format.

```
meals_server.py 1, U X
code > meals_server.py > ...

118
119 @mcp.tool()
120 def random_meal():
121     try:
122         response = httpx.get(
123             f"{BASE_URL}/random.php",
124             timeout=10.0,
125         )
126         response.raise_for_status()
127         data = response.json()
128     except (httpx.HTTPError, ValueError) as error:
129         raise RuntimeError(f"TheMealDB request failed: {error}") from error
130
131     meals = data.get("meals")
132
133     if not meals:
134         return {
135             "message": "no matches",
136             "result": None,
137         }
138
139     return format_meal_details(meals[0])
```

The screenshot displays the MCP Inspector web application. The browser's address bar shows the URL: `127.0.0.1:6274/?MCP_INSPECTOR_API_TOKEN=c1f147579bf3d550d125bdb7773b93e911c733aee9824ec3d5bd9a966b6dfa09`. The application has a dark theme and a top navigation bar with tabs for "Servers", "Tools", "Prompts", and "Resources".

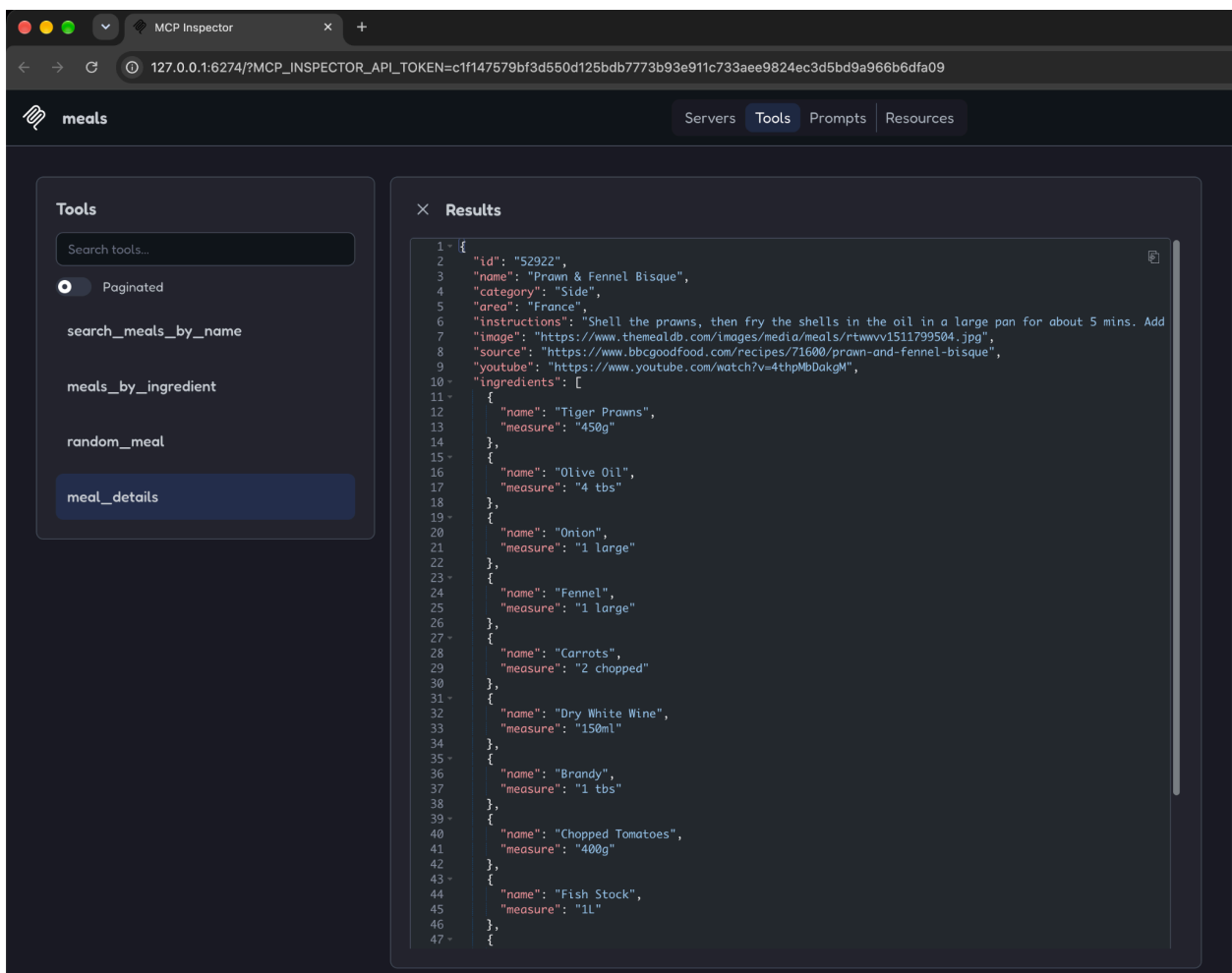
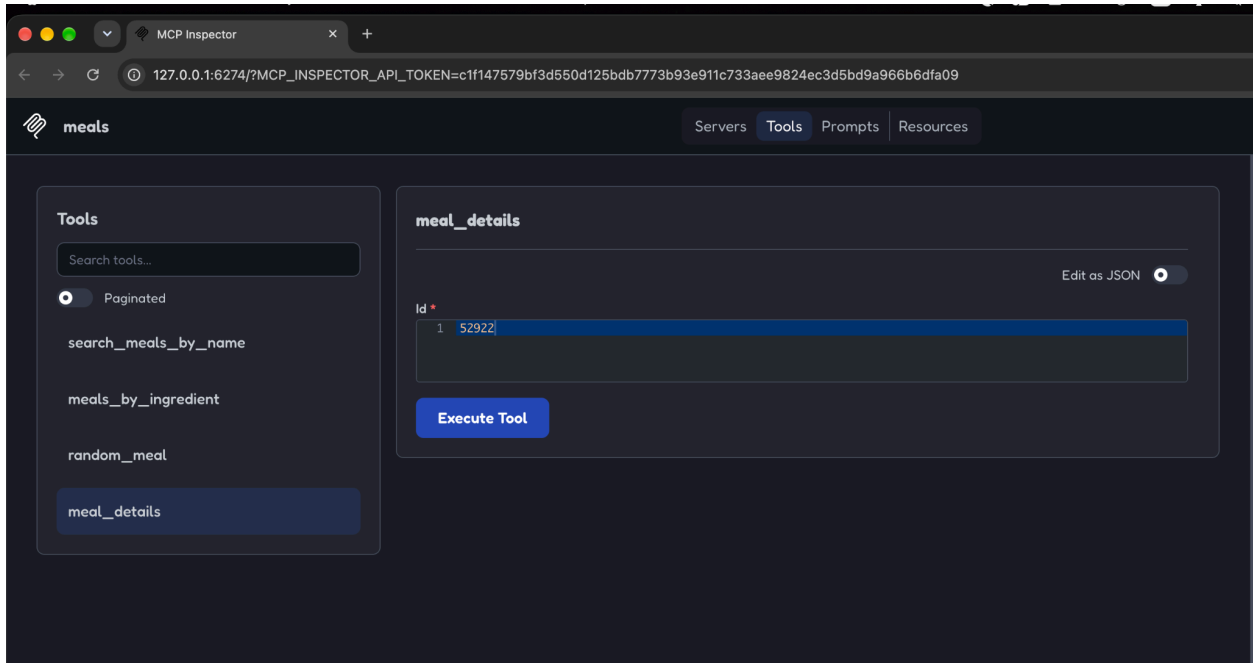
On the left, a "Tools" sidebar contains a search bar and a "Paginated" toggle. Below these are four tool buttons: `search_meals_by_name`, `meals_by_ingredient`, and `random_meal` (which is highlighted).

The main area on the right, titled "Results", displays a JSON object for a selected meal. The JSON data is as follows:

```
1 {
2   "id": "52922",
3   "name": "Prawn & Fennel Bisque",
4   "category": "Side",
5   "area": "France",
6   "instructions": "Shell the prawns, then fry the shells in the oil in a large pan for about 5 mins. Add
7   "image": "https://www.themeadb.com/images/media/meals/rtwvvv1511799504.jpg",
8   "source": "https://www.bbcgoodfood.com/recipes/71600/prawn-and-fennel-bisque",
9   "youtube": "https://www.youtube.com/watch?v=4thpMbDakgM",
10  "ingredients": [
11    {
12      "name": "Tiger Prawns",
13      "measure": "450g"
14    },
15    {
16      "name": "Olive Oil",
17      "measure": "4 tbs"
18    },
19    {
20      "name": "Onion",
21      "measure": "1 large"
22    },
23    {
24      "name": "Fennel",
25      "measure": "1 large"
26    },
27    {
28      "name": "Carrots",
29      "measure": "2 chopped"
30    },
31    {
32      "name": "Dry White Wine",
33      "measure": "150ml"
34    },
35    {
36      "name": "Brandy",
37      "measure": "1 tbs"
38    },
39    {
40      "name": "Chopped Tomatoes",
41      "measure": "400g"
42    },
43    {
44      "name": "Fish Stock",
45      "measure": "1l"
46    },
47    {
```

9. I implemented the meal_details MCP tool using FastMCP. The tool calls TheMealDB /lookup.php endpoint with the provided meal ID and returns the full recipe details for that meal. The response is passed through the shared format_meal_details helper so the output includes the meal ID, name, category, cuisine area, instructions, image, source URL, YouTube URL, and ingredient/measure pairs in a consistent schema. I tested the tool in MCP Inspector using meal ID 52922 and confirmed that the returned recipe details follow the required format.

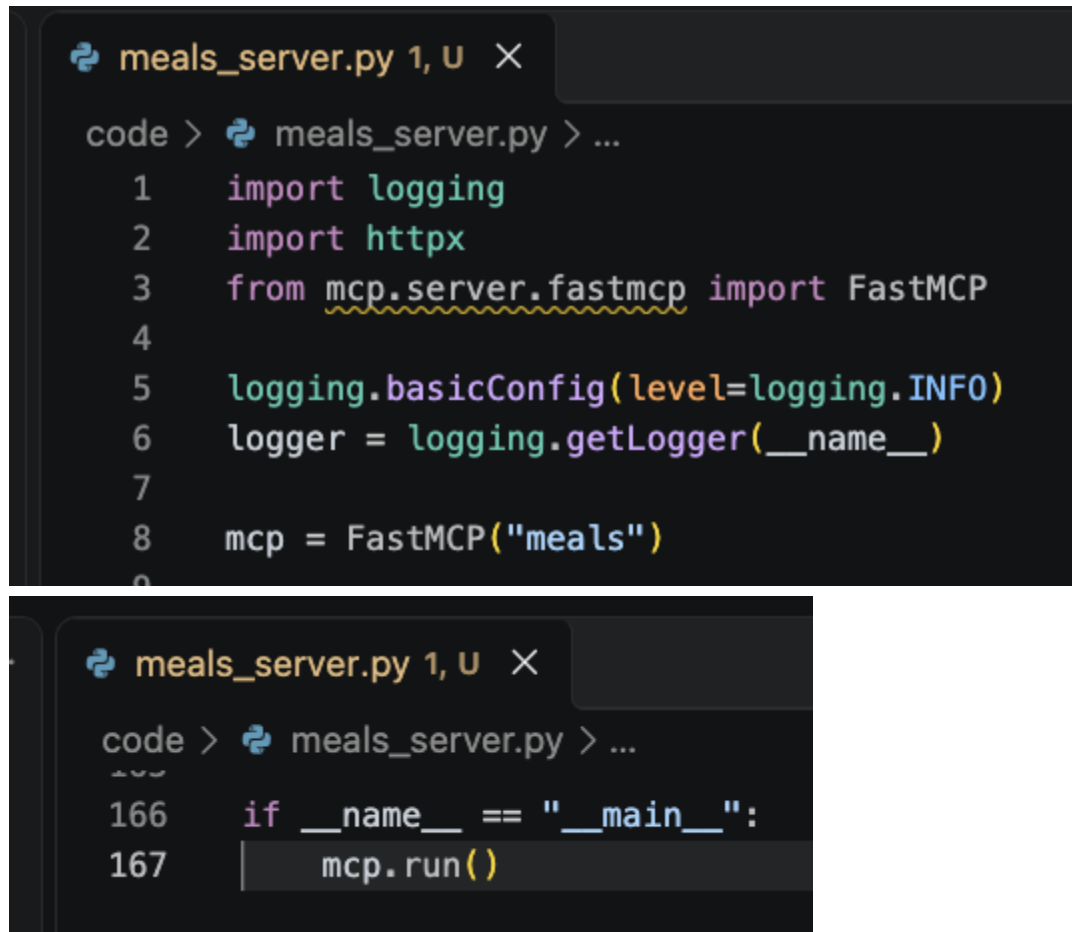
```
meals_server.py 1, U X
code > meals_server.py > ...
142 @mcp.tool()
143 def meal_details(id: str | int):
144     try:
145         response = httpx.get(
146             f"{BASE_URL}/lookup.php",
147             params={"i": str(id)},
148             timeout=10.0,
149         )
150         response.raise_for_status()
151         data = response.json()
152     except (httpx.HTTPError, ValueError) as error:
153         raise RuntimeError(f"TheMealDB request failed: {error}") from error
154
155     meals = data.get("meals")
156
157     if not meals:
158         return {
159             "message": "no matches",
160             "result": None,
161         }
162
163     return format_meal_details(meals[0])
164
```



10. I installed the MCP Python SDK with CLI support. Because the assignment implementation uses the FastMCP API, I used MCP 1.x and verified the installed version as 1.30.0.

```
(base) optim@Mac data260-9981 % pip show mcp
Name: mcp
Version: 1.30.0
Summary: Model Context Protocol SDK
Home-page: https://modelcontextprotocol.io
Author: Anthropic, PBC.
Author-email:
License: MIT
Location: /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages
Requires: anyio, httpx, httpx-sse, jsonschema, pydantic, pydantic-settings, pyjwt, python-multipart, sse-starlette, starlette, typing-extensions, typing-inspection, uvicorn
Required-by:
○ (base) optim@Mac data260-9981 %
```

11. I created a single Python MCP server file, code/meals_server.py, that defines all four TheMealDB tools using FastMCP. The server is named meals and is started with mcp.run(), which runs the MCP server over STDIO. I also configured logging through Python's logging module so diagnostic output goes to stderr instead of stdout, avoiding corruption of the MCP JSON-RPC stream.



The image contains two screenshots of a code editor. The top screenshot shows the first part of a Python file named `meals_server.py`. It includes imports for `logging` and `httpx`, and a `FastMCP` class from `mcp.server.fastmcp`. It also shows the configuration of a logger and the initialization of the `mcp` object with the name "meals". The bottom screenshot shows the `if __name__ == "__main__":` block, which calls `mcp.run()` to start the server.

```
meals_server.py 1, U X
code > meals_server.py > ...
1  import logging
2  import httpx
3  from mcp.server.fastmcp import FastMCP
4
5  logging.basicConfig(level=logging.INFO)
6  logger = logging.getLogger(__name__)
7
8  mcp = FastMCP("meals")
9
...
166 if __name__ == "__main__":
167     mcp.run()
```

12. I implemented the required input and output contract for `search_meals_by_name`. The tool accepts `query` as a string and `limit` as an integer restricted to the range 1–25. It returns a list of meal objects containing `id`, `name`, `area`, `category`, and `thumb`. I verified the contract in MCP Inspector using the query `Arrabiata` with a limit of 5.

meals_server.py 1, U X

```
code > meals_server.py > meals_by_ingredient
41 @mcp.tool()
42 def search_meals_by_name(query: str, limit: int = 5):
43     if limit < 1 or limit > 25:
44         raise ValueError("limit must be between 1 and 25")
45
46     try:
47         response = httpx.get(
48             f"{BASE_URL}/search.php",
49             params={"s": query},
50             timeout=10.0,
51         )
52         response.raise_for_status()
53         data = response.json()
54     except (httpx.HTTPError, ValueError) as error:
55         raise RuntimeError(f"TheMealDB request failed: {error}") from error
56
57     meals = data.get("meals")
58
59     if meals is None:
60         return {
61             "message": "no matches",
62             "results": [],
63         }
64
65     results = []
66
67     for meal in meals[:limit]:
68         results.append(
69             {
70                 "id": meal["idMeal"],
71                 "name": meal["strMeal"],
72                 "area": meal["strArea"],
73                 "category": meal["strCategory"],
74                 "thumb": meal["strMealThumb"],
75             }
76         )
77
78     return results
```

13. I implemented the required input and output contract for `meals_by_ingredient`. The tool accepts `ingredient` as a string and `limit` as an integer, then returns a list of meal objects containing `id`, `name`, and `thumb`. I verified the contract in MCP Inspector using `chicken` with a `limit` of 12.

```
meals_server.py 1, U X
code > meals_server.py > random_meal
81 @mcp.tool()
82 def meals_by_ingredient(ingredient: str, limit: int = 12):
83     if limit < 1 or limit > 25:
84         raise ValueError("limit must be between 1 and 25")
85
86     try:
87         response = httpx.get(
88             f"{BASE_URL}/filter.php",
89             params={"i": ingredient},
90             timeout=10.0,
91         )
92         response.raise_for_status()
93         data = response.json()
94     except (httpx.HTTPError, ValueError) as error:
95         raise RuntimeError(f"TheMealDB request failed: {error}") from error
96
97     meals = data.get("meals")
98
99     if meals is None:
100         return {
101             "message": "no matches",
102             "results": [],
103         }
104
105     results = []
106
107     for meal in meals[:limit]:
108         results.append(
109             {
110                 "id": meal["idMeal"],
111                 "name": meal["strMeal"],
112                 "thumb": meal["strMealThumb"],
113             }
114         )
115
116     return results
```

14. I implemented the required input and output contract for meal_details. The tool accepts a meal id as either a string or integer. It returns one detailed meal object containing id, name, category, area, instructions, image, source, youtube, and an ingredients list containing name and measure pairs. I verified this contract in MCP Inspector using meal ID 52922.

meals_server.py 1, U X

code > meals_server.py > ...

```
142 @mcp.tool()
143 def meal_details(id: str | int):
144     try:
145         response = httpx.get(
146             f"{BASE_URL}/lookup.php",
147             params={"i": str(id)},
148             timeout=10.0,
149         )
150         response.raise_for_status()
151         data = response.json()
152     except (httpx.HTTPError, ValueError) as error:
153         raise RuntimeError(f"TheMealDB request failed: {error}") from error
154
155     meals = data.get("meals")
156
157     if not meals:
158         return {
159             "message": "no matches",
160             "result": None,
161         }
162
163     return format_meal_details(meals[0])
```

```

meals_server.py 1, U X
code > meals_server.py > ...
12
13 def format_meal_details(meal):
14     ingredients = []
15
16     for index in range(1, 21):
17         ingredient = meal.get(f"strIngredient{index}")
18         measure = meal.get(f"strMeasure{index}")
19
20         if ingredient and ingredient.strip():
21             ingredients.append(
22                 {
23                     "name": ingredient.strip(),
24                     "measure": measure.strip() if measure else "",
25                 }
26             )
27
28     return {
29         "id": meal["idMeal"],
30         "name": meal["strMeal"],
31         "category": meal["strCategory"],
32         "area": meal["strArea"],
33         "instructions": meal["strInstructions"],
34         "image": meal["strMealThumb"],
35         "source": meal.get("strSource"),
36         "youtube": meal.get("strYoutube"),
37         "ingredients": ingredients,
38     }

```

15. I implemented the required input and output contract for `random_meal`. The tool takes no input parameters and returns one randomly selected meal using the same detailed output structure as `meal_details`, including id, name, category, area, instructions, image, source, youtube, and an ingredients list containing name and measure pairs. If TheMealDB returns no meal data, the tool returns a clear empty result with the message "no matches". Network or JSON parsing errors are caught and raised as a clean runtime error so MCP Inspector can display the failure without crashing the server.

```

meals_server.py 1, U X
code > meals_server.py > ...
119 @mcp.tool()
120 def random_meal():
121     try:
122         response = httpx.get(
123             f"{BASE_URL}/random.php",
124             timeout=10.0,
125         )
126         response.raise_for_status()
127         data = response.json()
128     except (httpx.HTTPError, ValueError) as error:
129         raise RuntimeError(f"TheMealDB request failed: {error}") from error
130
131     meals = data.get("meals")
132
133     if not meals:
134         return {
135             "message": "no matches",
136             "result": None,
137         }
138
139     return format_meal_details(meals[0])

```

16 & 17. implemented error handling for all TheMealDB MCP tools and ran the server locally using MCP Inspector. If TheMealDB returns no meal data, the tools return a clear empty result with a "no matches" message. HTTP and JSON parsing errors are caught and raised as clean runtime errors so MCP Inspector can display the failure without crashing the server. I used MCP Inspector to execute and verify all four tools: search_meals_by_name, meals_by_ingredient, random_meal, and meal_details

B. Domain MCP Server

- I created a second MCP server for my s9981_rel Local Restaurant Inspections domain. The server defines exactly three tools: search_inspections for searching inspection records, inspection_detail for retrieving one inspection by ID, and restaurant_inspection_summary for aggregating inspection statistics for a restaurant.

domain_server.py 1, U X

code > domain_server.py > ...

```
31
32 @mcp.tool()
33 def search_inspections(query: str, limit: int = 10):
34     if not query.strip():
35         return error_response("query must not be empty")
36
37     if limit < 1 or limit > 25:
38         return error_response("limit must be between 1 and 25")
39
40     db = SessionLocal()
41
42     try:
43         inspections = (
44             db.query(RestaurantInspection)
45             .filter(
46                 or_(
47                     RestaurantInspection.restaurant_name.ilike(f"%{query}%"),
48                     RestaurantInspection.restaurant_address.ilike(f"%{query}%"),
49                     RestaurantInspection.inspection_code.ilike(f"%{query}%"),
50                 )
51             )
52             .limit(limit)
53             .all()
54         )
55
56         data = [
57             {
58                 "id": inspection.id,
59                 "restaurant_name": inspection.restaurant_name,
60                 "restaurant_address": inspection.restaurant_address,
61                 "inspection_code": inspection.inspection_code,
62                 "violation_count": inspection.violation_count,
63                 "restaurant_id": inspection.restaurant_id,
64             }
65             for inspection in inspections
66         ]
67
68         return success_response(data)
69
70     except Exception as error:
71         logger.error("search_inspections failed: %s", error)
72         return error_response("failed to search inspections")
73
74     finally:
75         db.close()
76
77
```

```

domain_server.py 1, U X
code > domain_server.py > ...
//
78 @mcp.tool()
79 def inspection_detail(inspection_id: int):
80     if inspection_id < 1:
81         return error_response("inspection_id must be a positive integer")
82
83     db = SessionLocal()
84
85     try:
86         inspection = (
87             db.query(RestaurantInspection)
88             .filter(RestaurantInspection.id == inspection_id)
89             .first()
90         )
91
92         if inspection is None:
93             return error_response(
94                 f"inspection {inspection_id} was not found"
95             )
96
97         data = {
98             "id": inspection.id,
99             "restaurant_name": inspection.restaurant_name,
100             "restaurant_address": inspection.restaurant_address,
101             "inspection_code": inspection.inspection_code,
102             "violation_count": inspection.violation_count,
103             "restaurant_id": inspection.restaurant_id,
104             "created_at": (
105                 inspection.created_at.isoformat()
106                 if inspection.created_at
107                 else None
108             ),
109             "updated_at": (
110                 inspection.updated_at.isoformat()
111                 if inspection.updated_at
112                 else None
113             ),
114         }
115
116         return success_response(data)
117
118     except Exception as error:
119         logger.error("inspection_detail failed: %s", error)
120         return error_response("failed to retrieve inspection details")
121
122     finally:
123         db.close()
124

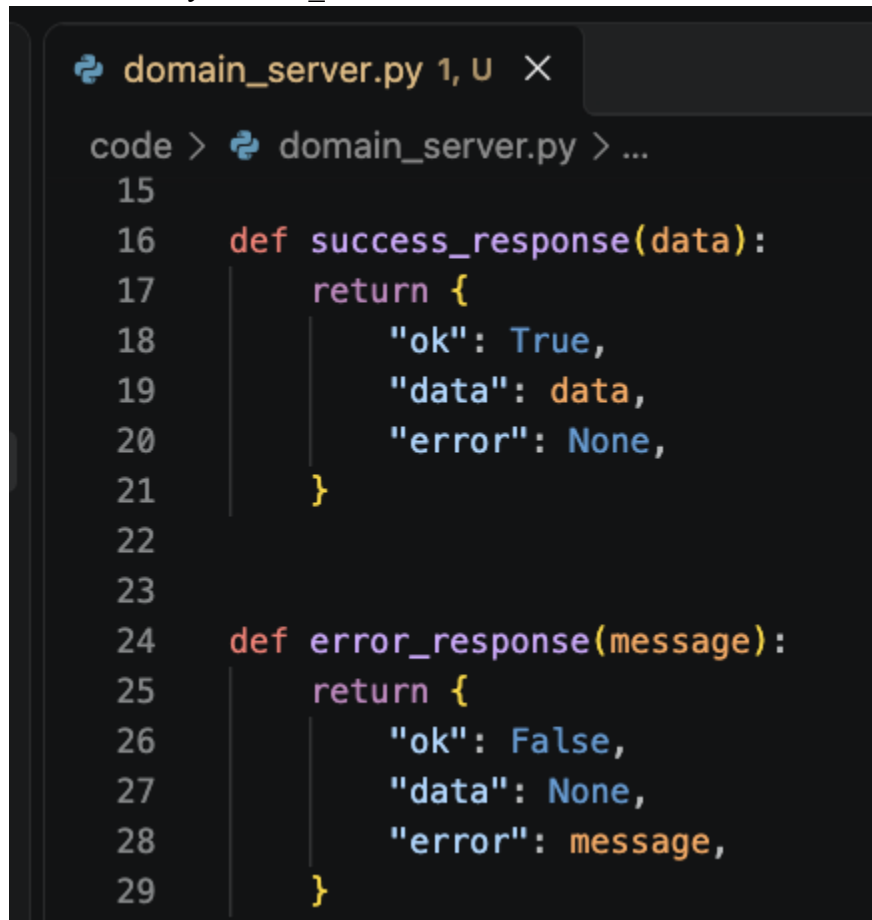
```

```

domain_server.py 1, U X
code > domain_server.py > ...
124
125
126 @mcp.tool()
127 def restaurant_inspection_summary(restaurant_id: int):
128     if restaurant_id < 1:
129         return error_response("restaurant_id must be a positive integer")
130
131     db = SessionLocal()
132
133     try:
134         result = (
135             db.query(
136                 func.count(RestaurantInspection.id),
137                 func.sum(RestaurantInspection.violation_count),
138                 func.avg(RestaurantInspection.violation_count),
139                 func.max(RestaurantInspection.violation_count),
140             )
141             .filter(RestaurantInspection.restaurant_id == restaurant_id)
142             .one()
143         )
144
145         inspection_count = result[0]
146
147         if inspection_count == 0:
148             return error_response(
149                 f"no inspections found for restaurant {restaurant_id}"
150             )
151
152         data = {
153             "restaurant_id": restaurant_id,
154             "inspection_count": inspection_count,
155             "total_violations": int(result[1] or 0),
156             "average_violations": float(result[2] or 0),
157             "maximum_violations": int(result[3] or 0),
158         }
159
160         return success_response(data)
161
162     except Exception as error:
163         logger.error(
164             "restaurant_inspection_summary failed: %s",
165             error,
166         )
167         return error_response(
168             "failed to calculate restaurant inspection summary"
169         )
170
171     finally:
172         db.close()
173

```

- I defined a shared response envelope for all three tools using the format {ok, data, error}. Successful calls return ok: true, the result in data, and error: null. Invalid or failed calls return ok: false, data: null, and a descriptive error message. This same response format will be reused later by `execute_tool` in Part 4.

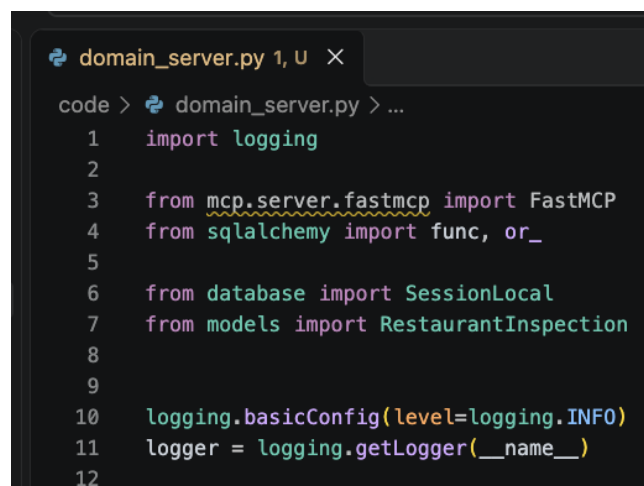


```

domain_server.py 1, U X
code > domain_server.py > ...
15
16 def success_response(data):
17     return {
18         "ok": True,
19         "data": data,
20         "error": None,
21     }
22
23
24 def error_response(message):
25     return {
26         "ok": False,
27         "data": None,
28         "error": message,
29     }

```

- The MCP server uses Python logging for diagnostic messages. Because Python logging writes to stderr by default, MCP STUDIO output is not written to stdout, which prevents corruption of the JSON-RPC stream.



```

domain_server.py 1, U X
code > domain_server.py > ...
1 import logging
2
3 from mcp.server.fastmcp import FastMCP
4 from sqlalchemy import func, or_
5
6 from database import SessionLocal
7 from models import RestaurantInspection
8
9
10 logging.basicConfig(level=logging.INFO)
11 logger = logging.getLogger(__name__)
12

```

127.0.0.1:6274/?MCP_INSPECTOR_API_TOKEN=766778376d3604f1dba3b3acaae8d8cca79f993318deb0bad034f54ae9983c6

s9981-domain Servers Tools Prompts Resources

Tools

Search tools...

☒ Paginated

search_inspections

inspection_detail

restaurant_inspection_summary

search_inspections

Query * Restaurant

Limit 5

Execute Tool

Edit as JSON ☒

127.0.0.1:6274/?MCP_INSPECTOR_API_TOKEN=766778376d3604f1dba3b3acaae8d8cca79f993318deb0bad034f54ae9983c6

s9981-domain Servers Tools Prompts Resources

Tools

Search tools...

☒ Paginated

search_inspections

inspection_detail

restaurant_inspection_summary

Results

```
1- {
2  "ok": true,
3  "data": [
4    {
5      "id": 10016,
6      "restaurant_name": "Restaurant 12",
7      "restaurant_address": "112 Test Street",
8      "inspection_code": "INSP-10016",
9      "violation_count": 0,
10     "restaurant_id": 12
11   },
12   {
13     "id": 10017,
14     "restaurant_name": "Restaurant 13",
15     "restaurant_address": "113 Test Street",
16     "inspection_code": "INSP-10017",
17     "violation_count": 0,
18     "restaurant_id": 13
19   },
20   {
21     "id": 10018,
22     "restaurant_name": "Restaurant 14",
23     "restaurant_address": "114 Test Street",
24     "inspection_code": "INSP-10018",
25     "violation_count": 0,
26     "restaurant_id": 14
27   },
28   {
29     "id": 10019,
30     "restaurant_name": "Restaurant 15",
31     "restaurant_address": "115 Test Street",
32     "inspection_code": "INSP-10019",
33     "violation_count": 0,
34     "restaurant_id": 15
35   },
36   {
37     "id": 10020,
38     "restaurant_name": "Restaurant 16",
39     "restaurant_address": "116 Test Street",
40     "inspection_code": "INSP-10020",
41     "violation_count": 0,
42     "restaurant_id": 16
43   },
44   ],
45   "error": null
46 }
```

127.0.0.1:6274/?MCP_INSPECTOR_API_TOKEN=766778376d3604f1dba3b3acaae8d8cca79f993318deb0bad034f54ae9983c6

s9981-domain

ServersToolsPromptsResources

Tools

Search tools...

Paginated

search_inspections

inspection_detail

restaurant_inspection_summary

search_inspections

Edit as JSON

Query *
Restaurant

Limit
0

Execute Tool

s9981-domain

ServersToolsPromptsResources

Tools

Search tools...

Paginated

search_inspections

inspection_detail

restaurant_inspection_summary

Results

```
1 {  
2   "ok": false,  
3   "data": null,  
4   "error": "limit must be between 1 and 25"  
5 }
```

The image displays two screenshots of the MCP Inspector web interface, which is used for managing and interacting with Model Context Protocol (MCP) servers. The interface is dark-themed and includes a navigation bar at the top with tabs for Servers, Tools, Prompts, and Resources.

Top Screenshot: The 'Tools' tab is active. On the left, a 'Tools' panel shows a search bar and a list of tools: 'search_inspections', 'inspection_detail' (highlighted), and 'restaurant_inspection_summary'. On the right, a 'Results' panel displays a JSON response from the 'inspection_detail' tool:

```
1 {  
2   "ok": false,  
3   "data": null,  
4   "error": "inspection 10006 was not found"  
5 }
```

Bottom Screenshot: The 'Tools' tab is still active. The 'Tools' panel on the left remains the same. The main area on the right now shows the configuration for the 'inspection_detail' tool. It includes a text input for 'Inspection Id' with the value '10006' and an 'Execute Tool' button. An 'Edit as JSON' toggle is also visible in the top right corner of the tool configuration area.

The image shows two screenshots of the MCP Inspector web interface, demonstrating the configuration and execution of the 'inspection_detail' tool.

Top Screenshot: The interface shows the 'Tools' panel on the left with a search bar and a list of tools: 'search_inspections', 'inspection_detail' (selected), and 'restaurant_inspection_summary'. The 'inspection_detail' tool configuration panel on the right has a title 'inspection_detail', an 'Edit as JSON' toggle, and an 'Inspection Id' input field with the value '0'. An 'Execute Tool' button is visible at the bottom of the configuration panel.

Bottom Screenshot: The interface shows the same 'Tools' panel. The 'inspection_detail' tool configuration panel is now closed, and a 'Results' panel is open, displaying the JSON response from the tool execution:

```
1 {  
2   "ok": false,  
3   "data": null,  
4   "error": "inspection_id must be a positive integer"  
5 }
```

The image shows two screenshots of a web application interface for 's9981-domain'. The top screenshot shows the 'Tools' section with a search bar and a list of tools: 'search_inspections', 'inspection_detail', and 'restaurant_inspection_summary'. The 'restaurant_inspection_summary' tool is selected, and its configuration is shown on the right. It has a 'Restaurant Id' field with the value '1' and an 'Execute Tool' button. The bottom screenshot shows the same interface after the tool has been executed. The 'Results' panel on the right displays a JSON response:

```
1 {
2   "ok": true,
3   "data": {
4     "restaurant_id": 1,
5     "inspection_count": 1,
6     "total_violations": 2,
7     "average_violations": 2,
8     "maximum_violations": 2
9   },
10  "error": null
11 }
```

127.0.0.1:6274/?MCP_INSPECTOR_API_TOKEN=766778376d3604f1dba3b3acaae8d8cca79f993318deb0bad034f54ae9983c6

s9981-domain Servers Tools Prompts Resources

Tools

Search tools...

Paginated

search_inspections

inspection_detail

restaurant_inspection_summary

restaurant_inspection_summary

Restaurant Id *

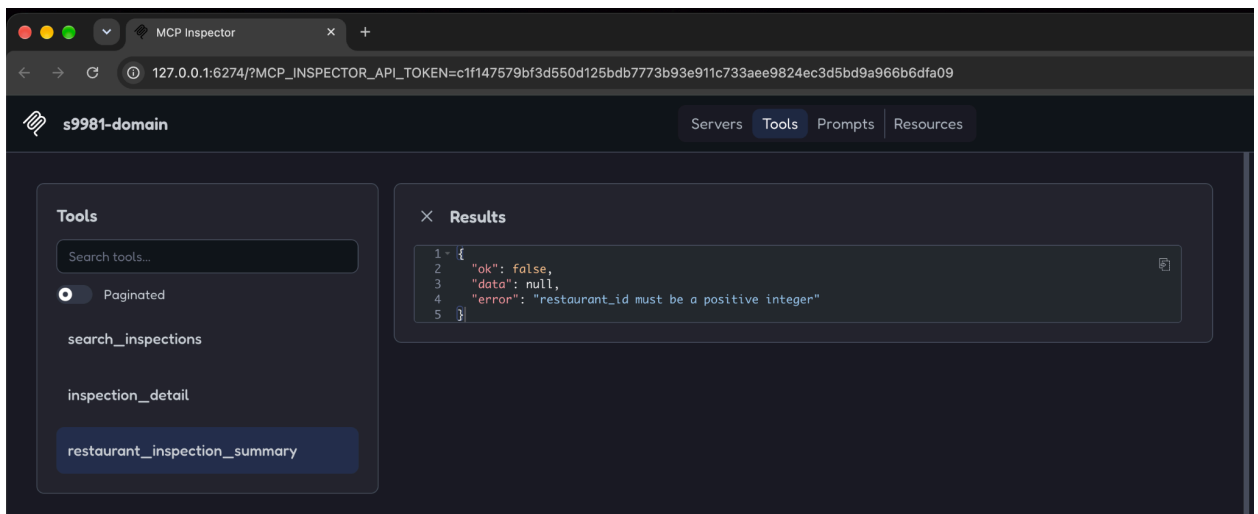
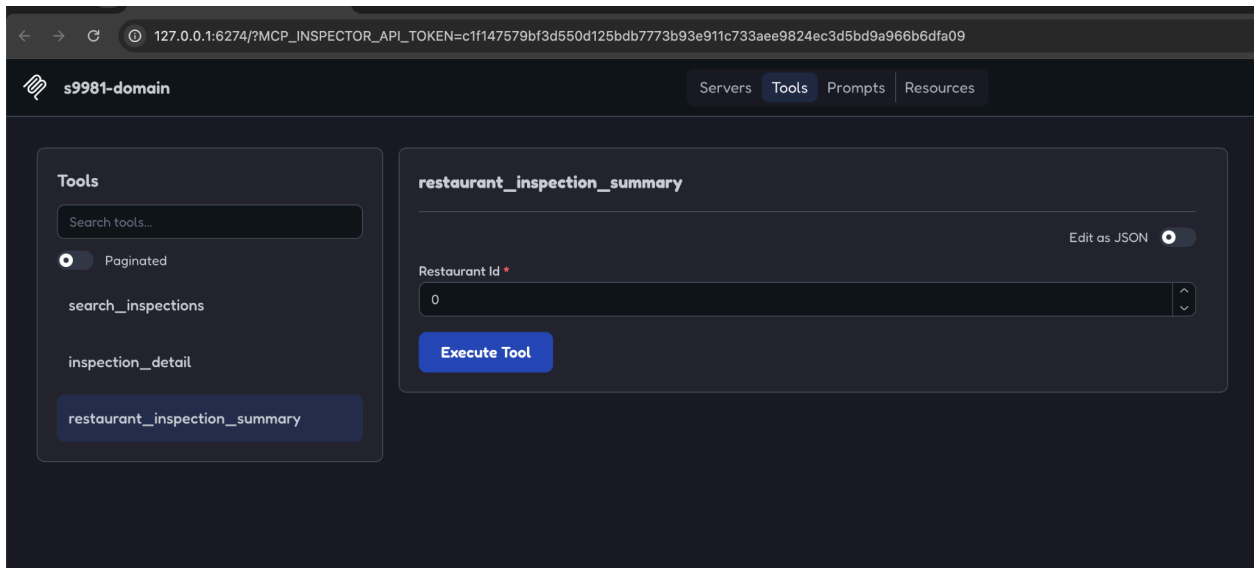
1

Execute Tool

Edit as JSON

Results

```
1 {
2   "ok": true,
3   "data": {
4     "restaurant_id": 1,
5     "inspection_count": 1,
6     "total_violations": 2,
7     "average_violations": 2,
8     "maximum_violations": 2
9   },
10  "error": null
11 }
```



Part 3 - Tool Contracts Under Stress

18.

search_inspections

Expected JSON input schema:

```
JSON
{
  "query": "string",
  "limit": "integer from 1 to 25"
}
```

Rejected JSON input:

```
JSON
{
  "query": "Restaurant",
  "limit": 0
}
```

Returned error output:

```
JSON
{
  "ok": false,
  "data": null,
  "error": "limit must be between 1 and 25"
}
```

The call was rejected because limit = 0 is outside the allowed range of 1–25.

inspection_detail

Expected JSON input schema:

```
JSON
{
  "inspection_id": "positive integer"
}
```

Rejected JSON input:

```
JSON
{
  "inspection_id": 0
}
```

Returned error output:

```
JSON
{
  "ok": false,
  "data": null,
  "error": "inspection_id must be a positive integer"
}
```

The call was rejected because `inspection_id = 0` is not a positive integer.

restaurant_inspection_summary

Expected JSON input schema:

```
JSON
{
  "restaurant_id": "positive integer"
}
```

Rejected JSON input:

```
JSON
{
  "restaurant_id": 0
}
```

Returned error output:

```
JSON
{
  "ok": false,
  "data": null,
  "error": "restaurant_id must be a positive integer"
}
```

The call was rejected because `restaurant_id = 0` is not a positive integer.

19.

I implemented a bounded retry helper for storage operations with a maximum of three attempts, a 2.0-second timeout per attempt, and exponential backoff delays starting at 0.10 seconds and doubling on each retry up to a bounded maximum. I used controlled failure injection to demonstrate the required behaviors.

In the first case, the database operation succeeded on the first attempt.

In the second case, the first attempt was intentionally failed and the operation succeeded on the second attempt after a 0.10-second backoff.

In the third case, all three attempts were intentionally failed. After exhausting the allowed retries, the helper returned a clean error result with `ok: false`, `data: null`, and an error message instead of crashing.

```
Debug Console  Terminal  Ports  zsh + v
9981 % python code/retry_demo.py
CASE 1: Success on first attempt
{
  "ok": true,
  "data": {
    "message": "database operation succeeded",
    "inspection_count": 4991
  },
  "error": null,
  "attempts": 1
}

CASE 2: First attempt fails, retry succeeds
WARNING:domain_server:Attempt 1 failed: simulated transient failure on attempt 1.
Retrying in 0.10 seconds.
{
  "ok": true,
  "data": {
    "message": "database operation succeeded",
    "inspection_count": 4991
  },
  "error": null,
  "attempts": 2
}

CASE 3: All attempts fail
WARNING:domain_server:Attempt 1 failed: simulated transient failure on attempt 1.
Retrying in 0.10 seconds.
WARNING:domain_server:Attempt 2 failed: simulated transient failure on attempt 2.
Retrying in 0.20 seconds.
{
  "ok": false,
  "data": null,
  "error": "simulated transient failu
o (base) optim@Mac data260-9981 %
```

20.

I used `VERIFY_SEED = 269981` to reproducibly simulate transient failures at injected failure rates of 0%, 20%, and 50%. For each rate, I ran 50 calls using the same bounded retry policy from item 19, for a total of 150 calls. Each call recorded whether it succeeded, the number of attempts, latency in milliseconds, and any final error. The raw call records were saved to `reports/hw05/raw/fault_injection_calls.csv`.

The 0% failure-rate condition achieved a 100.0% success rate with a mean latency of 2.305 ms and a p99 latency of 3.079 ms. The 20% condition also achieved a 100.0% success rate, but retry activity increased the mean latency to 39.858 ms and the p99 latency to 312.882 ms. At a 50% injected failure rate, the success rate decreased to 86.0%, with a mean latency of 85.292 ms and a p99 latency of 312.372 ms.

Injected failure rate	Success rate	Mean latency (ms)	p99 latency (ms)
0%	100.0%	2.305	3.079
20%	100.0%	39.858	312.882
50%	86.0%	85.292	312.372

```

● (base) optim@Mac data260-9981 % python code/failure_experiment.py
Student: Timothy Song
VERIFY_SEED: 269981

Running failure rate: 0%
Success rate: 100.0%
Mean latency: 2.305 ms
p99 latency: 3.079 ms

Running failure rate: 20%
WARNING:domain_server:Attempt 1 failed: simulated transient failure on attempt 1.
  Retrying in 0.10 seconds.
WARNING:domain_server:Attempt 2 failed: simulated transient failure on attempt 2.
  Retrying in 0.20 seconds.
WARNING:domain_server:Attempt 1 failed: simulated transient failure on attempt 1.
  Retrying in 0.10 seconds.
WARNING:domain_server:Attempt 1 failed: simulated transient failure on attempt 1.
  Retrying in 0.10 seconds.
WARNING:domain_server:Attempt 1 failed: simulated transient failure on attempt 1.
  Retrying in 0.10 seconds.
WARNING:domain_server:Attempt 2 failed: simulated transient failure on attempt 2.
  Retrying in 0.20 seconds.
WARNING:domain_server:Attempt 1 failed: simulated transient failure on attempt 1.
  Retrying in 0.10 seconds.
WARNING:domain_server:Attempt 1 failed: simulated transient failure on attempt 1.
  Retrying in 0.10 seconds.
WARNING:domain_server:Attempt 1 failed: simulated transient failure on attempt 1.
  Retrying in 0.10 seconds.
WARNING:domain_server:Attempt 2 failed: simulated transient failure on attempt 2.
  Retrying in 0.20 seconds.
WARNING:domain_server:Attempt 1 failed: simulated transient failure on attempt 1.
  Retrying in 0.10 seconds.
WARNING:domain_server:Attempt 1 failed: simulated transient failure on attempt 1.
  Retrying in 0.10 seconds.
WARNING:domain_server:Attempt 2 failed: simulated transient failure on attempt 2.
  Retrying in 0.20 seconds.
WARNING:domain_server:Attempt 1 failed: simulated transient failure on attempt 1.
  Retrying in 0.10 seconds.
Success rate: 100.0%
Mean latency: 39.858 ms
p99 latency: 312.882 ms

Running failure rate: 50%
WARNING:domain_server:Attempt 1 failed: simulated transient failure on attempt 1.
  Retrying in 0.10 seconds.
WARNING:domain_server:Attempt 2 failed: simulated transient failure on attempt 2.
  Retrying in 0.20 seconds.
WARNING:domain_server:Attempt 1 failed: simulated transient failure on attempt 1.
  Retrying in 0.10 seconds.
WARNING:domain_server:Attempt 1 failed: simulated transient failure on attempt 1.
  Retrying in 0.10 seconds.
WARNING:domain_server:Attempt 2 failed: simulated transient failure on attempt 2.
  Retrying in 0.20 seconds.
WARNING:domain_server:Attempt 1 failed: simulated transient failure on attempt 1.
  Retrying in 0.10 seconds.
WARNING:domain_server:Attempt 2 failed: simulated transient failure on attempt 2.
  Retrying in 0.20 seconds.
WARNING:domain_server:Attempt 1 failed: simulated transient failure on attempt 1.
  Retrying in 0.10 seconds.
WARNING:domain_server:Attempt 2 failed: simulated transient failure on attempt 2.
  Retrying in 0.20 seconds.
WARNING:domain_server:Attempt 1 failed: simulated transient failure on attempt 1.
  Retrying in 0.10 seconds.
WARNING:domain_server:Attempt 1 failed: simulated transient failure on attempt 1.
  Retrying in 0.10 seconds.
WARNING:domain_server:Attempt 2 failed: simulated transient failure on attempt 2.
  Retrying in 0.20 seconds.
WARNING:domain_server:Attempt 1 failed: simulated transient failure on attempt 1.
  Retrying in 0.10 seconds.
WARNING:domain_server:Attempt 1 failed: simulated transient failure on attempt 1.
  Retrying in 0.10 seconds.

```

[illegible]

21.

Based on the experiment results, the retry policy is suitable for an interactive assistant under low to moderate transient failure rates. At 0% injected failures, all calls succeeded with very low latency. At a 20% failure rate, the retry policy still achieved a 100% success rate, although the mean latency increased from 2.305 ms to 39.858 ms and the p99 latency increased to 312.882 ms because some calls required retries and backoff delays. At a 50% injected failure rate, the success rate decreased to 86.0%, showing that the bounded three-attempt policy cannot recover from every high-failure sequence.

For an interactive assistant, I would keep the retry count relatively small and the backoff delays short so the user does not experience excessive response latency. The current maximum of three attempts with short exponential backoff provides a reasonable balance between reliability and responsiveness.

For batch processing, where longer execution times are acceptable, I would increase the maximum number of retries and allow longer timeout and backoff values. For example, I could increase the maximum attempts from 3 to 5, increase the timeout per attempt, and use a larger bounded exponential backoff. This would improve the probability of eventually completing operations during temporary storage failures, while the additional latency would be less important in a non-interactive workload.

Part 4 - One Safe Tool Entry Point and Offline Tests**22.**

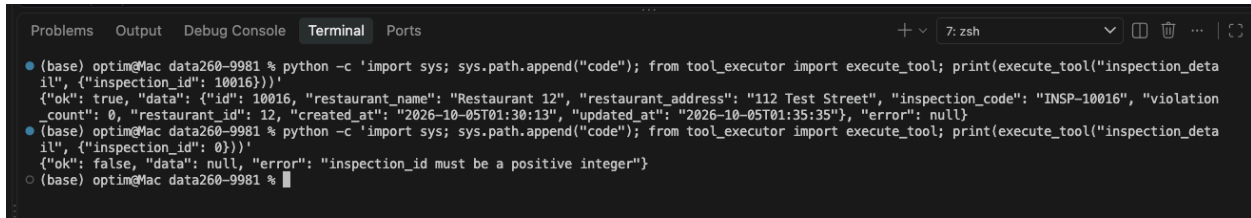
I created `execute_tool(name, inputs)` as the single entry point that the Part 5 agent will use to call any of the three domain tools. The function maps the requested tool name to `search_inspections`, `inspection_detail`, or `restaurant_inspection_summary`, executes the selected tool with the provided input object, and returns the result as a JSON string using the same `{ok, data, error}` response envelope defined in Part 2B. Invalid tool names, invalid input objects, argument errors, and unexpected exceptions are caught and returned as clean error responses instead of crashing the program.

I verified the function with a valid `inspection_detail` call using inspection ID 10016, which returned `ok: true`, the inspection data, and `error: null`. I also verified an invalid call using inspection ID 0, which returned `ok: false`, `data: null`, and the validation error `"inspection_id must be a positive integer"`.

```

tool_executor.py U
code > tool_executor.py > ...
10 def execute_tool(name, inputs):
11     try:
12         if not isinstance(name, str) or not name.strip():
13             result = {
14                 "ok": False,
15                 "data": None,
16                 "error": "tool name must be a non-empty string",
17             }
18             return json.dumps(result)
19
20         if not isinstance(inputs, dict):
21             result = {
22                 "ok": False,
23                 "data": None,
24                 "error": "inputs must be a JSON object",
25             }
26             return json.dumps(result)
27
28         tools = {
29             "search_inspections": search_inspections,
30             "inspection_detail": inspection_detail,
31             "restaurant_inspection_summary":
32                 restaurant_inspection_summary,
33         }
34
35         tool = tools.get(name)
36
37         if tool is None:
38             result = {
39                 "ok": False,
40                 "data": None,
41                 "error": f"unknown tool: {name}",
42             }
43             return json.dumps(result)
44
45         result = tool(*inputs)
46
47         if not isinstance(result, dict):
48             result = {
49                 "ok": False,
50                 "data": None,
51                 "error": "tool returned an invalid response",
52             }
53
54         return json.dumps(result)
55
56     except TypeError as error:
57         result = {
58             "ok": False,
59             "data": None,
60             "error": f"invalid tool inputs: {error}",
61         }
62
63         return json.dumps(result)
64
65     except Exception as error:
66         result = {
67             "ok": False,
68             "data": None,
69             "error": f"tool execution failed: {error}",
70         }
71
72         return json.dumps(result)

```



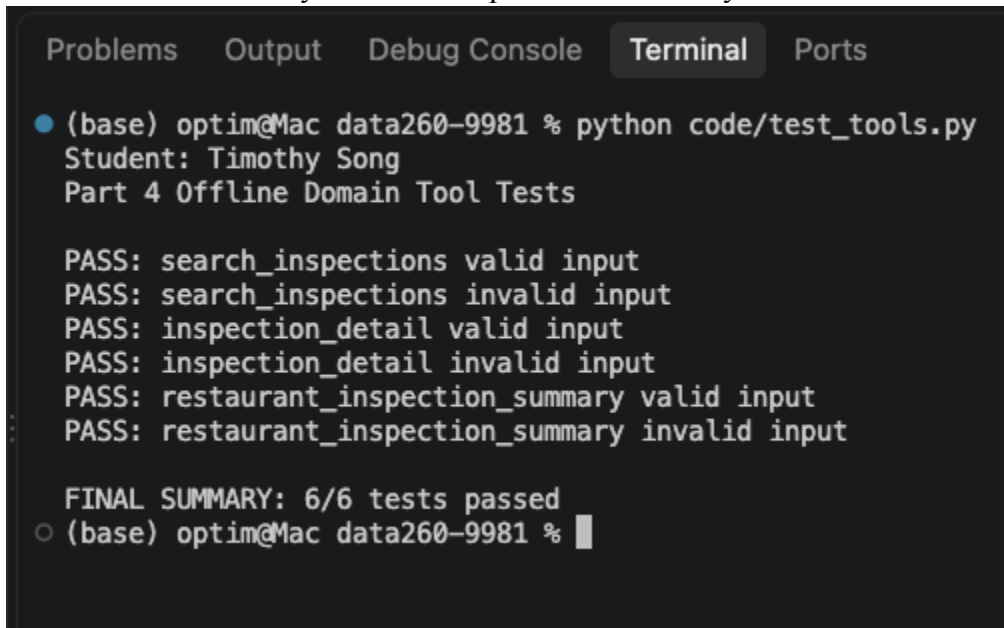
```

Problems Output Debug Console Terminal Ports
+ v 7: zsh
• (base) optim@Mac data260-9981 % python -c 'import sys; sys.path.append("code"); from tool_executor import execute_tool; print(execute_tool("inspection_detail", {"inspection_id": 10016}))'
{"ok": true, "data": {"id": 10016, "restaurant_name": "Restaurant 12", "restaurant_address": "112 Test Street", "inspection_code": "INSP-10016", "violation_count": 0, "restaurant_id": 12, "created_at": "2026-10-05T01:30:13", "updated_at": "2026-10-05T01:35:35"}, "error": null}
• (base) optim@Mac data260-9981 % python -c 'import sys; sys.path.append("code"); from tool_executor import execute_tool; print(execute_tool("inspection_detail", {"inspection_id": 0}))'
{"ok": false, "data": null, "error": "inspection_id must be a positive integer"}
○ (base) optim@Mac data260-9981 %

```

23.

I created a Python test runner using assert statements to validate the three domain tools through `execute_tool`. For each tool, I included one valid-input test and one invalid-input test using the same rejected cases documented in Part 3. The runner prints PASS or FAIL for each individual test and a final summary. All six tests passed successfully.



```

Problems Output Debug Console Terminal Ports
• (base) optim@Mac data260-9981 % python code/test_tools.py
Student: Timothy Song
Part 4 Offline Domain Tool Tests

PASS: search_inspections valid input
PASS: search_inspections invalid input
PASS: inspection_detail valid input
PASS: inspection_detail invalid input
PASS: restaurant_inspection_summary valid input
PASS: restaurant_inspection_summary invalid input

FINAL SUMMARY: 6/6 tests passed
○ (base) optim@Mac data260-9981 %

```

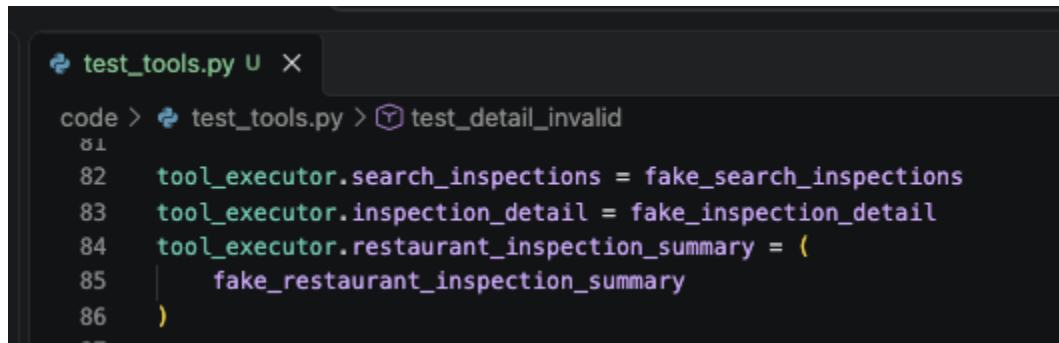
```

test_tools.py U X
code > test_tools.py > ...
110 def test_search_valid():
111     result = json.loads(
112         tool_executor.execute_tool(
113             "search_inspections",
114             {
115                 "query": "Restaurant",
116                 "limit": 5,
117             },
118         )
119     )
120
121     assert result["ok"] is True
122     assert result["error"] is None
123     assert isinstance(result["data"], list)
124
125
126 def test_search_invalid():
127     result = json.loads(
128         tool_executor.execute_tool(
129             "search_inspections",
130             {
131                 "query": "Restaurant",
132                 "limit": 0,
133             },
134         )
135     )
136
137     assert result["ok"] is False
138     assert result["data"] is None
139     assert result["error"] == "limit must be between 1 and 25"
140
141
142 def test_detail_valid():
143     result = json.loads(
144         tool_executor.execute_tool(
145             "inspection_detail",
146             {
147                 "inspection_id": 1,
148             },
149         )
150     )
151
152     assert result["ok"] is True
153     assert result["error"] is None
154     assert result["data"]["id"] == 1
155
156
157 def test_detail_invalid():
158     result = json.loads(
159         tool_executor.execute_tool(
160             "inspection_detail",
161             {
162                 "inspection_id": 0,
163             },
164         )
165     )
166
167     assert result["ok"] is False
168     assert result["data"] is None
169     assert (
170         result["error"]
171         == "inspection_id must be a positive integer"
172     )
173
174
175 def test_summary_valid():
176     result = json.loads(
177         tool_executor.execute_tool(
178             "restaurant_inspection_summary",
179             {
180                 "restaurant_id": 1,
181             },
182         )
183     )
184
185     assert result["ok"] is True
186     assert result["error"] is None
187     assert result["data"]["restaurant_id"] == 1
188
189
190 def test_summary_invalid():
191     result = json.loads(
192         tool_executor.execute_tool(
193             "restaurant_inspection_summary",
194             {
195                 "restaurant_id": 0,
196             },
197         )
198     )
199
200     assert result["ok"] is False
201     assert result["data"] is None
202     assert (
203         result["error"]
204         == "restaurant_id must be a positive integer"
205     )
206

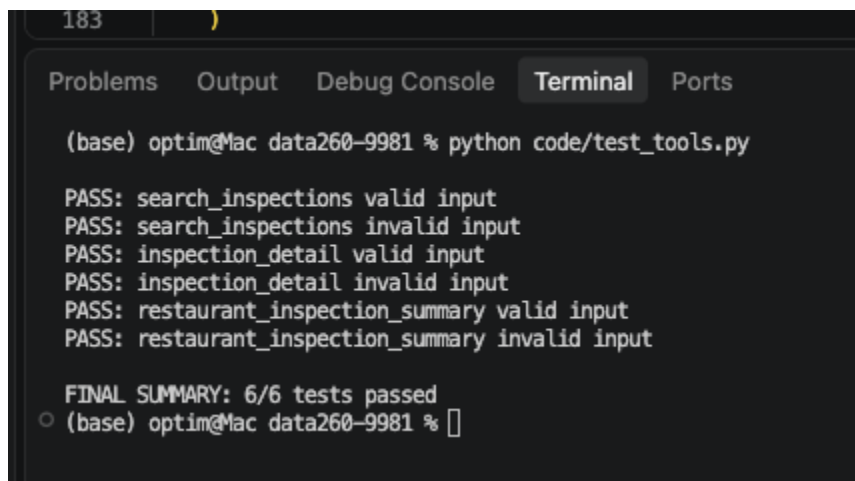
```

24.

I used dependency injection so the Part 4 tests can run offline and repeatedly without requiring the live MySQL database, an external API, or an LLM. In `test_tools.py`, I created deterministic fake implementations of `search_inspections`, `inspection_detail`, and `restaurant_inspection_summary`, then replaced the live tool references used by `execute_tool` with these fake functions during testing. This allows the same valid and invalid behaviors to be tested without external dependencies. The offline test runner completed successfully with all 6 tests passing.



```
test_tools.py U X
code > test_tools.py > test_detail_invalid
81
82 tool_executor.search_inspections = fake_search_inspections
83 tool_executor.inspection_detail = fake_inspection_detail
84 tool_executor.restaurant_inspection_summary = (
85     fake_restaurant_inspection_summary
86 )
87
```



```
183 )
Problems Output Debug Console Terminal Ports
(base) optim@Mac data260-9981 % python code/test_tools.py

PASS: search_inspections valid input
PASS: search_inspections invalid input
PASS: inspection_detail valid input
PASS: inspection_detail invalid input
PASS: restaurant_inspection_summary valid input
PASS: restaurant_inspection_summary invalid input

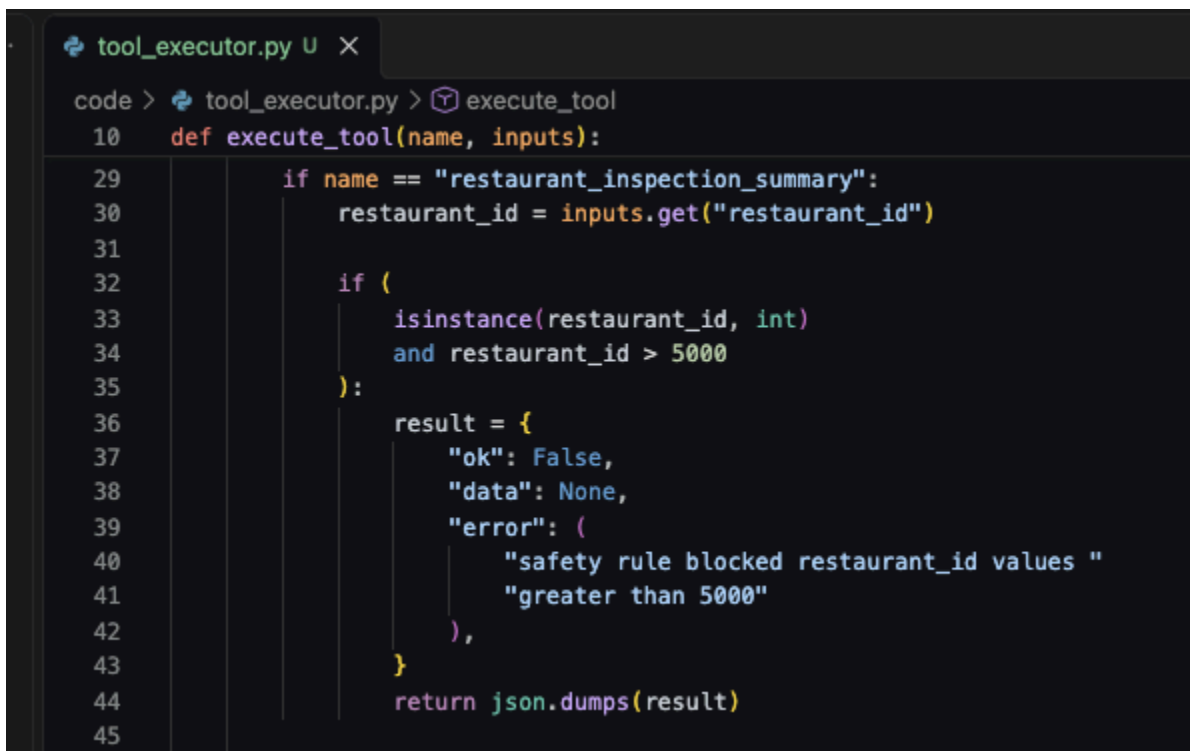
FINAL SUMMARY: 6/6 tests passed
(base) optim@Mac data260-9981 %
```

Part 5 - Agent Loop, Safety Rule, and Reflection

I. Safety Rule

I added a domain-specific safety rule inside `execute_tool` for the `restaurant_inspection_summary` operation. The rule blocks summary requests when `restaurant_id` is greater than 5000, preventing implausible out-of-range restaurant identifiers from being passed to the underlying tool. When the rule is violated, `execute_tool` returns the standard `{ok, data, error}` response with `ok: false`, `data: null`, and a descriptive error message instead of raising an exception.

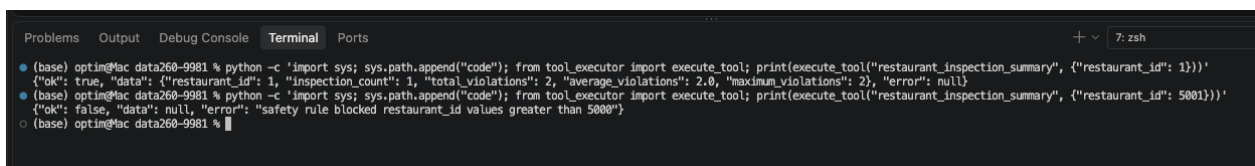
I demonstrated an allowed call using `restaurant_id = 1`, which successfully returned the restaurant inspection summary. I also demonstrated a blocked call using `restaurant_id = 5001`, which returned the safety-rule error without executing the domain operation or crashing.



```

tool_executor.py U X
code > tool_executor.py > execute_tool
10 def execute_tool(name, inputs):
29     if name == "restaurant_inspection_summary":
30         restaurant_id = inputs.get("restaurant_id")
31
32         if (
33             isinstance(restaurant_id, int)
34             and restaurant_id > 5000
35         ):
36             result = {
37                 "ok": False,
38                 "data": None,
39                 "error": (
40                     "safety rule blocked restaurant_id values "
41                     "greater than 5000"
42                 ),
43             }
44             return json.dumps(result)
45

```



```

Problems Output Debug Console Terminal Ports
+ 7: zsh
• (base) optin@mac data260-9981 % python -c 'import sys; sys.path.append("code"); from tool_executor import execute_tool; print(execute_tool("restaurant_inspection_summary", {"restaurant_id": 1}))'
{"ok": true, "data": {"restaurant_id": 1, "inspection_count": 1, "total_violations": 2, "average_violations": 2.0, "maximum_violations": 2}, "error": null}
• (base) optin@mac data260-9981 % python -c 'import sys; sys.path.append("code"); from tool_executor import execute_tool; print(execute_tool("restaurant_inspection_summary", {"restaurant_id": 5001}))'
{"ok": false, "data": null, "error": "safety rule blocked restaurant_id values greater than 5000"}
○ (base) optin@mac data260-9981 %

```

II. Agent Loop

I implemented `run_agent(user_input)` using the local Ollama model `qwen3:8b` and the three domain tools. The agent decides whether to call `search_inspections`, `inspection_detail`, or `restaurant_inspection_summary`, and all tool execution is routed through `execute_tool`. I added a step counter and a `max_steps` limit to prevent unbounded execution. Each agent step is appended to `reports/hw05/raw/agent_runs.jsonl`, including the step number, requested tool, tool input, returned result, and final stop reason.

I tested the agent with the prompt “Find Restaurant 12 and tell me about one of its inspections.” On step 1, the agent called `restaurant_inspection_summary` for `restaurant_id = 12`. The tool returned one inspection with zero violations. On step 2, the agent returned the final answer and stopped with `normal_completion`.

```

agent.py U X
code > agent.py > run_agent
9
10 MODEL = "qwen3:8b"
11 OLLAMA_URL = "http://localhost:11434/api/chat"
12 LOG_FILE = "reports/hw05/raw/agent_runs.jsonl"
13
14 TOOLS = [
15     "search_inspections",
16     "inspection_detail",
17     "restaurant_inspection_summary",
18 ]
19

```

```

agent.py U X
code > agent.py > run_agent
85
86 def call_ollama(messages):
87     response = requests.post(
88         OLLAMA_URL,
89         json={
90             "model": MODEL,
91             "messages": messages,
92             "stream": False,
93             "format": "json",
94             "think": False,
95         },
96         timeout=60,
97     )
98
99     response.raise_for_status()
100
101     data = response.json()
102     content = data["message"]["content"]
103
104     return json.loads(content)
105

```

```

agent.py U X
code > agent.py > run_agent
110 def run_agent(user_input, max_steps=5):
111     messages = [
112         {
113             "role": "system",
114             "content": SYSTEM_PROMPT,
115         },
116         {
117             "role": "user",
118             "content": user_input,
119         },
120     ]
121
122     for step in range(1, max_steps + 1):
123         try:
124             model_output = call_ollama(messages)
125

```

```

agent.py U X
code > agent.py > run_agent
107 def run_agent(user_input, max_steps=5):
195
196     tool_name = model_output.get("tool")
197     tool_inputs = model_output.get(
198         "inputs",
199         {},
200     )
201
202     if tool_name not in TOOLS:
203         result = json.dumps(
204             {
205                 "ok": False,
206                 "data": None,
207                 "error": {
208                     f"unknown tool requested by model: "
209                     f"{tool_name}"
210                 },
211             }
212         )
213     else:
214         result = execute_tool(
215             tool_name,
216             tool_inputs,
217         )
218
219     result_object = json.loads(result)
220
221     write_log(
222         {
223             "step": step,
224             "user_input": user_input,
225             "model_output": model_output,
226             "tool_call": tool_name,
227             "tool_input": tool_inputs,
228             "tool_result": result_object,
229             "stop_reason": None,
230         }
231     )
232

```

```

Problems Output Debug Console Terminal Ports
+-----+
(base) optima@calvin:~/src % python config.py
Enter a restaurant inspection question: Fine Restaurant 12 and tell me about one of its inspections.
{
  "summary": "Restaurant 12 has 1 inspection record, and there are no violations reported in that inspection.",
  "steps": 2,
  "info_reason": "normal_completion"
}

(base) optima@calvin:~/src % tail -n 5 reports/Full/ragent/ops.json
{"step": 1, "user_input": "Find Restaurant 12 and tell me about one of its inspections.", "model_output": {"action": "tool", "tool": "restaurant_inspection_summary", "parameters": {"restaurant_id": 122}, "tool_result": {"ok": true, "data": {"restaurant_id": 12, "inspects": [{"inspection_id": 122, "summary": "Restaurant 12 has 1 inspection record, and there are no violations reported in that inspection."}], "info_reason": "normal_completion"}}, "timestamp": "2020-10-07T18:35:13.420+08:00"},
{"step": 2, "user_input": "Find Restaurant 12 and tell me about one of its inspections.", "model_output": {"action": "finish", "summary": "Restaurant 12 has 1 inspection record, and there are no violations reported in that inspection."}, "tool_call": null, "tool_result": null, "final_answer": "Restaurant 12 has 1 inspection record, and there are no violations reported in that inspection.", "info_reason": "normal_completion", "timestamp": "2020-10-07T18:35:13.484+08:00"}
(base) optima@calvin:~/src %

```

III. Additional Tests

I extended the existing Part 4 offline test suite with two Part 5 tests. The first verifies that `execute_tool` blocks a `restaurant_inspection_summary` request when `restaurant_id = 5001`, returning the expected safety-rule error envelope. The second uses a deterministic `MockModel` with `run_agent` and verifies that the agent stops after reaching `max_steps = 3` with the stop reason `max_steps`.

These tests run offline without an API key, live model, external API, or database. The domain tools are replaced with deterministic fake implementations, and the agent test injects MockModel instead of calling Ollama. The complete suite passed all 8 tests.

```
code > test_tools.py > ...
85
86
87 class MockModel:
88     def generate(self, messages):
89         return {
90             "action": "tool",
91             "tool": "search_inspections",
92             "inputs": {
93                 "query": "Restaurant",
94                 "limit": 5,
95             },
96         }
97
98
99 passed = 0
100 total = 0
101
```

```

test_tools.py U X
code > test_tools.py > test_detail_valid
218
219 def test_safety_rule_blocked():
220     result = json.loads(
221         tool_executor.execute_tool(
222             "restaurant_inspection_summary",
223             {
224                 "restaurant_id": 5001,
225             },
226         )
227     )
228
229     assert result["ok"] is False
230     assert result["data"] is None
231     assert (
232         result["error"]
233         == "safety rule blocked restaurant_id values greater than 5000"
234     )
235
236
237 def test_agent_max_steps():
238     mock_model = MockModel()
239
240     result = run_agent(
241         "Keep searching for restaurant inspections.",
242         max_steps=3,
243         model=mock_model,
244     )
245
246     assert result["answer"] is None
247     assert result["steps"] == 3
248     assert result["stop_reason"] == "max_steps"
249     assert (
250         result["error"]
251         == "maximum number of agent steps reached"
252     )
253
254

```

```

Problems  Output  Debug Console  Terminal  Ports
• (base) optim@Mac data260-9981 % python code/test_tools.py
Student: Timothy Song
Part 4 + Part 5 Offline Domain Tool Tests

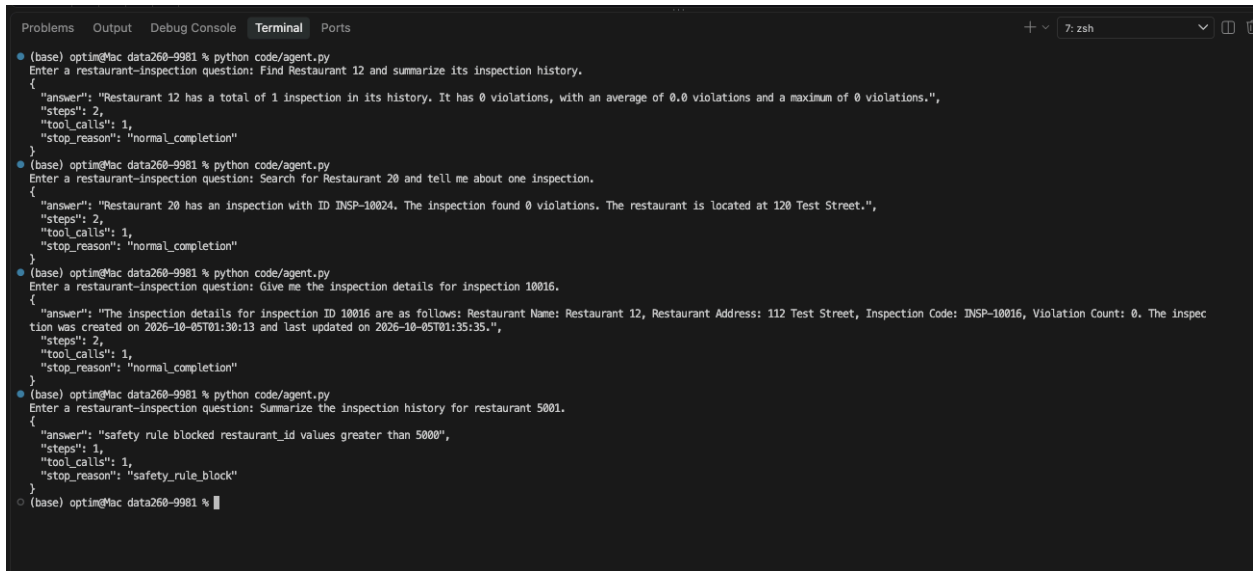
PASS: search_inspections valid input
PASS: search_inspections invalid input
PASS: inspection_detail valid input
PASS: inspection_detail invalid input
PASS: restaurant_inspection_summary valid input
PASS: restaurant_inspection_summary invalid input
PASS: execute_tool safety rule blocked
PASS: run_agent MockModel max_steps

FINAL SUMMARY: 8/8 tests passed
○ (base) optim@Mac data260-9981 %

```

IV. Metrics

I ran four scenarios using the local Ollama qwen3:8b model and recorded the step count, tool-call count, and final stop reason for each run. The first three scenarios completed normally in two steps with one tool call each. The fourth scenario requested a summary for restaurant_id = 5001, which violated the domain safety rule and stopped after one step with the stop reason `safety_rule_block`.



```

(base) optin@Mac data260-9981 % python code/agent.py
Enter a restaurant-inspection question: Find Restaurant 12 and summarize its inspection history.
{
  "answer": "Restaurant 12 has a total of 1 inspection in its history. It has 0 violations, with an average of 0.0 violations and a maximum of 0 violations.",
  "steps": 2,
  "tool_calls": 1,
  "stop_reason": "normal_completion"
}
(base) optin@Mac data260-9981 % python code/agent.py
Enter a restaurant-inspection question: Search for Restaurant 20 and tell me about one inspection.
{
  "answer": "Restaurant 20 has an inspection with ID INSP-10024. The inspection found 0 violations. The restaurant is located at 120 Test Street.",
  "steps": 2,
  "tool_calls": 1,
  "stop_reason": "normal_completion"
}
(base) optin@Mac data260-9981 % python code/agent.py
Enter a restaurant-inspection question: Give me the inspection details for inspection 10016.
{
  "answer": "The inspection details for inspection ID 10016 are as follows: Restaurant Name: Restaurant 12, Restaurant Address: 112 Test Street, Inspection Code: INSP-10016, Violation Count: 0. The inspection was created on 2026-10-05T01:30:13 and last updated on 2026-10-05T01:35:35.",
  "steps": 2,
  "tool_calls": 1,
  "stop_reason": "normal_completion"
}
(base) optin@Mac data260-9981 % python code/agent.py
Enter a restaurant-inspection question: Summarize the inspection history for restaurant 5001.
{
  "answer": "safety rule blocked restaurant_id values greater than 5000",
  "steps": 1,
  "tool_calls": 1,
  "stop_reason": "safety_rule_block"
}
(base) optin@Mac data260-9981 %
  
```

Scenario	Prompt	Step Count	Tool-Call Count	Stop Reason
1	Find Restaurant 12 and summarize its inspection history.	2	1	normal_completion
2	Search for Restaurant 20 and tell me about one inspection.	2	1	normal_completion
3	Give me the inspection details for inspection 10016.	2	1	normal_completion
4	Summarize the inspection history for restaurant 5001.	1	1	safety_rule_block

V. Reflection

I selected the agent run for the prompt, “Find Restaurant 12 and summarize its inspection history.” This run completed normally in two steps with one tool call. On the first step, the local Ollama model qwen3:8b examined the user request and decided that it needed structured inspection data before producing an answer. It selected the `restaurant_inspection_summary` tool and supplied `restaurant_id = 12` as the input.

The harness then routed that request through `execute_tool`, which applied the normal validation and safety logic before calling the underlying domain tool. The tool returned a successful response showing that Restaurant 12 had one inspection, zero total violations, an average of 0.0 violations, and a maximum of zero violations. The harness logged the step number, tool name, tool input, tool result, and intermediate stop reason to `agent_runs.jsonl`.

After receiving the tool result, the harness added the result back into the conversation context and asked the model to decide whether another tool call was necessary or whether it could provide a final response. On step two, the model determined that the returned summary contained enough information to answer the user. It therefore produced a final natural-language response instead of requesting another tool.

The harness recorded the final answer and stopped with the reason `normal_completion`. The run used two total steps and one tool call. This run demonstrates how the harness constrains the model to use verified tool output before answering questions about the restaurant-inspection database.

AI USE

1. What I used an AI assistant for and what I did myself

I used an AI assistant mainly for implementation guidance, debugging support, test planning, and report organization/grammar. The assistant helped me break the homework into smaller steps, suggested code structure for the MCP servers, retry helper, `'execute_tool'`, agent loop, offline tests, metrics collection, repository artifacts, and other sections I was stuck on. It also helped me identify which relevant screenshots and outputs were needed for the report. In addition, it helped me generate repository files based on work I had already completed and verified.

I performed the actual implementation, ran the code locally, tested the FastAPI application, Redux client, MCP Inspector tools, failure-injection experiments, offline test suite, and Ollama agent scenarios, and verified the resulting outputs. I also created the final report, selected the screenshots, and organized the repository files.

2. One AI-produced output that was wrong or unsuitable, or one thing I independently verified

One thing I independently verified was the dependency setup for the MCP servers. The initial project requirements did not include all packages needed by the new MCP implementation. In particular, I verified that ``httpx`` and the MCP CLI dependency were required for the server code and development workflow.

3. How I detected the problem or verified the result

I detected the issue by running the MCP server and checking the installed package environment. I also verified the MCP version with ``pip show mcp`` and confirmed that the FastMCP implementation worked with MCP 1.30.0. I then ran the server through MCP Inspector and successfully executed the meal and domain tools, which confirmed that the dependency configuration was correct.

4. What I changed and why it works now

I updated ``requirements.txt`` to include:

- ``httpx``
- ``mcp[cli]<2``

This matches the implementation because the MCP servers use ``httpx`` for HTTP requests and FastMCP from MCP 1.x. After making the change, the servers started correctly in MCP Inspector and the required tools executed successfully. This resolved the missing dependency issue and made the environment reproducible for the submitted repository.